

Suivi multi-objets aux changements d'apparence dans des vidéos fisheye FiTAC-MOT

Rapport de stage

soutenu le 30 juin 2026

par

Xihao Wang

Encadrant entreprise : Dr. Yassir Zardoua, Dr. Rania OTHMAN, Dr. Joelle JREIS

Encadrant universitaire : M. Nicolas Lutz

Caplogy Innovation

Remerciements

En premier lieu, je tiens à remercier mes responsables de stage. Un grand merci pour leur accueil au sein de l'entreprise, ainsi que pour leur confiance lors de cette aventure dans le monde professionnel et leurs précieux conseils. Je les remercie aussi pour leur disponibilité et la qualité de leur encadrement en entreprise.

Je remercie également toute l'équipe R&D à *Caplogy* pour son accueil, son esprit d'équipe et ses conseils qui m'ont aidé à réussir ce travail.

Je souhaite aussi remercier particulièrement mes responsables du Master à l'Université de Montpellier, qui m'ont fourni les outils théoriques et pratiques nécessaires au bon déroulement de ce stage.

J'adresse mes profonds remerciements au personnel du Master *Informatique parcours IMAGINE* de l'Université de Montpellier pour son soutien et la qualité de l'enseignement offert.

Enfin, j'exprime mes vifs remerciements aux membres du jury pour l'intérêt qu'ils ont porté à mon travail en acceptant de l'examiner et de l'enrichir avec leurs remarques.

Ce stage m'a permis d'affiner certaines pistes pour bâtir mon projet d'orientation professionnelle et signer l'aboutissement de mon diplôme universitaire.

Dédicaces

*Je dédie ce travail à
Mon père et ma mère, mon grand frère et mon petit frère,
Toute ma famille,
Ainsi qu'à mon compagnon, mes amis et mes collègues,
Qui m'ont toujours soutenu et accompagné.
Sans votre soutien, je n'aurais pas pu poursuivre jusqu'à aujourd'hui mon parcours d'études
et de recherche en France.
Merci de m'avoir encouragé jusqu'à l'aboutissement de ce stage.*

Table des matières

Acronymes

Résumé	1
Abstract	2
Présentation de l'entreprise	3
Introduction générale	4
1 Introduction	4
2 Contexte général	4
3 Motivation et Objectifs	5
4 Organisation du rapport	6
État de l'art	8
1 Introduction	8
2 Travaux connexes	9
2.1 Suivi multi-objets par détection	9
2.2 Représentation d'apparence pour la ré-identification	11
2.3 Modélisation de l'identité par mémoire	13
3 Conclusion	16
Contribution	17
1 Introduction	17
2 Configuration de référence de type DeepSORT avec vecteurs d'apparence OSNet	18
2.1 Principe suivi par détection de la configuration de référence	18

2.2	Extraction des vecteurs d'apparence : OSNet pour la configuration de référence et FastReID pour le modèle temporel	19
2.3	Association détection-trajectoire : Kalman, coût d'apparence et association hongroise	21
3	TemporalAttentionScorer avec vecteurs d'apparence FastReID	22
3.1	Vue d'ensemble du TemporalAttentionScorer	23
3.2	Architecture du TemporalAttentionScorer	24
3.3	Apprentissage supervisé du score temporel	29
3.4	Intégration du score dans l'association existante	30
3.5	Règle d'acceptation avec récupération temporelle	31
3.6	Post-traitement AFLink	32
4	Intégration dans le pipeline fisheye/gnomonique	33
4.1	Position du traqueur dans le pipeline global	34
4.2	Adaptation des détections et extraction ReID dans le pipeline fisheye	34
5	Conclusion	34
Expérimentations et résultats		36
1	Introduction	36
2	Données et métriques	36
2.1	Jeux de données	36
2.2	Métriques d'évaluation	39
3	Préparation des données et configurations	41
3.1	Annotation de CapFiCA	41
3.2	Configurations évaluées	43
3.3	L'entraînement du modèle temporel appris	46
3.4	Étude de la mémoire à long terme	47
4	Expérimentations	48
4.1	Expérience 1 : étude d'ablation du modèle temporel	49
4.2	Expérience 2 : choix des encodeurs d'apparence	51
4.3	Expérience 3 : comparaison avec d'autres modèles de suivi	53
4.4	Expérience 4 : généralisation à d'autres scènes	54
4.5	Évaluation complémentaire avec boîtes orientées	56
5	Visualisation qualitative des résultats	57
6	Conclusion	58

Conclusion générale	59
7 Conclusion	59
8 Perspectives	60
Bibliographie	
Annexes	
9 Planning synthétique du stage	

Acronymes

TABLE 1 – Liste des acronymes utilisés dans le rapport.

Acronyme	Signification
AFLink	Appearance-Free Link, liaison de fragments de trajectoire sans apparence.
AssA	Association Accuracy, qualité de l'association des trajectoires.
BCE	Binary Cross-Entropy, perte binaire utilisée pour classer les paires positives et négatives.
BoT	Bag of Tricks, méthode de référence en ré-identification.
BoT-SORT	Méthode de suivi combinant mouvement, apparence et compensation caméra.
CapFiCA	Caplogy Fisheye Change Appearance, jeu de données construit dans ce travail.
DetA	Detection Accuracy, qualité des détections associées à la vérité terrain.
DeepSORT	Extension de SORT avec caractéristiques de ré-identification.
FastReID	Cadre de ré-identification utilisé pour la branche temporelle.
FiTAC-MOT	Fisheye Temporal Appearance-Change Multi-Object Tracking, méthode proposée.
FN	False Negative, objet réel non détecté.
FP	False Positive, détection ne correspondant pas à un objet réel.
FPS	Frames Per Second, nombre de trames traitées par seconde.
GT	Ground Truth, annotation de référence.
HOTA	Higher Order Tracking Accuracy, métrique combinant détection et association.
ID	Identity, identifiant associé à une personne suivie.
IDF1	Identity F1 Score, cohérence entre identités prédites et réelles.
IDFN	Identity False Negative, identité réelle non correctement retrouvée.
IDFP	Identity False Positive, identité prédite incorrectement attribuée.
IDSW	Identity Switch, changement d'identité pendant le suivi.
IDTP	Identity True Positive, identité correctement associée.
InfoNCE	Information Noise-Contrastive Estimation, perte de classement utilisée pour favoriser le bon candidat parmi plusieurs associations.
IoU	Intersection over Union, recouvrement entre deux boîtes englobantes.
LTM	Long-Term Memory, mémoire à long terme.
MLP	Multi-Layer Perceptron, réseau entièrement connecté.
MOT	Multi-Object Tracking, suivi multi-objets.
MOTA	Multiple Object Tracking Accuracy, métrique globale du suivi.

Acronyme	Signification
OSNet	Omni-Scale Network, encodeur d'apparence utilisé dans la référence.
ReID	Person Re-Identification, ré-identification de personnes.
SORT	Simple Online and Realtime Tracking, suivi fondé sur le mouvement.
STM	Short-Term Memory, mémoire à court terme.
StrongSORT	Extension de DeepSORT avec ré-identification renforcée.
TrackEval	Outil d'évaluation des métriques de suivi multi-objets.
TP	True Positive, détection correctement associée à un objet réel.
WEPDTOF	Jeu de données de suivi de piétons avec caméras fisheye.
YOLO	You Only Look Once, famille de détecteurs d'objets en temps réel.

Résumé

Dans les scènes de surveillance intelligente, le suivi stable de l'identité des piétons est une tâche importante. Ce problème devient plus difficile dans les vidéos fisheye, où l'apparence d'une personne peut varier fortement selon sa position dans l'image et son comportement au cours du temps. Les changements de vêtements, d'accessoires, de posture ou d'orientation du corps modifient les caractéristiques visuelles utilisées pour l'association. Ces variations peuvent alors provoquer des changements d'identité pendant le suivi.

Pour répondre à ce problème, nous proposons FiTAC-MOT, une méthode de suivi multi-objets adaptée aux changements d'apparence des piétons dans des vidéos fisheye. Elle suit un cadre de type *suivi par détection*. Testée sur différents jeux de données, la méthode montre une meilleure robustesse dans les scènes fisheye contenant des changements d'apparence et fournit des résultats de suivi plus stables pour des applications de surveillance intelligente.

Mots-clés : suivi multi-objets, caméra fisheye, changement d'apparence, association d'identités, mémoire temporelle, module spatio-temporel, FiTAC-MOT.

Abstract

In smart surveillance scenarios, reliably tracking pedestrian identities is a critical task. This problem becomes more challenging in fisheye videos, where a person’s appearance can vary significantly depending on their position in the image and their behavior over time. Changes in clothing, accessories, posture, or body orientation alter the visual features used for matching. These variations can then lead to identity changes during tracking.

To address this problem, we propose FiTAC-MOT, a multi-object tracking method adapted to changes in pedestrians’ appearance in fisheye videos. It follows a *tracking-by-detection* framework. Tested on various datasets, the method demonstrates greater robustness in fisheye scenes containing appearance changes and provides more stable tracking results for intelligent surveillance applications.

Keywords : multi-object tracking, fisheye camera, appearance change, identity association, temporal memory, spatiotemporal module, FiTAC-MOT.

Présentation de l'entreprise

Caplogy est un acteur en transformation digitale et en prestations dans le secteur de l'éducation et de l'enseignement supérieur. L'entreprise accompagne ses collaborateurs en France et à l'international dans les domaines Digital, IoT, Data/IA, Cybersécurité, DevOps/Cloud. Outre ses prestations d'enseignement, Caplogy propose un pôle R&D dans le but de développer en interne plusieurs projets d'innovation lui permettant de valoriser les compétences de ses collaborateurs, de développer leurs savoir-faire ainsi que d'être à la pointe de l'innovation dans des domaines cibles.

Les activités de recherche de l'équipe R&D de Caplogy sont multidisciplinaires et visent principalement à comprendre le comportement des personnes ; à traiter et analyser les données collectées de leur vie privée (par exemple, la localisation des individus, les signes vitaux des patients) ; à sécuriser ces données et à optimiser leur partage. Les données privées sont de nature spatio-temporelle et peuvent conduire à la divulgation d'une grande variété d'informations sensibles en matière de vie privée concernant les individus, telles que les routines et les habitudes, l'état de santé, les affiliations politiques/religieuses, les préférences et les activités effectuées.

Dans ce cadre, l'équipe R&D développe de nouvelles approches permettant d'améliorer la modélisation des données de la vie privée, optimiser le partage et le stockage d'une manière sécurisée et de prédire les cyberattaques d'une manière rapide et autonome. Les applications possibles incluent : la santé, les villes intelligentes, les maisons intelligentes, l'e-commerce et la finance.

Des projets de recherche divers sont en cours de développement dans le pôle R&D de Caplogy, dont PICO-DEV, KenetIQ, NAUTILUS et ICAPS. Parmi eux, PICO-DEV est lié à la prédiction des trajectoires avec caméra fisheye, ce qui correspond au contexte de ce stage consacré au suivi de piétons et à la robustesse face aux changements d'apparence.

Introduction générale

1 Introduction

Ce rapport présente le travail réalisé dans le cadre de mon stage au sein du pôle R&D de Caplogy. Le sujet porte sur le suivi de piétons dans des vidéos fisheye, avec une attention particulière portée à la stabilité des identifiants lorsque l'apparence des personnes varie au cours du temps.

L'objectif du stage est d'étudier et de développer des mécanismes permettant de rendre le suivi plus robuste dans ce contexte. Le travail s'appuie sur un pipeline existant de détection de piétons en images fisheye, puis se concentre principalement sur la partie association du suivi multi-objets.

Dans cette introduction générale, nous présentons d'abord le contexte du projet. Nous expliquons ensuite les motivations et les objectifs du stage, avant de décrire l'organisation du rapport.

2 Contexte général

Ce stage s'inscrit dans un projet de drone autonome équipé d'une caméra fisheye. L'objectif général est de pouvoir observer une scène large et d'y détecter des piétons pour des applications de surveillance intelligente. Le choix d'une caméra fisheye est intéressant dans ce contexte, car elle permet de couvrir une zone plus étendue qu'une caméra perspective classique, ce qui est utile pour un système embarqué comme un drone. En contrepartie, les images obtenues sont fortement déformées. Un piéton peut donc apparaître avec une taille ou une forme différente selon sa position dans l'image, en particulier lorsqu'il se trouve près des bords. Cette propriété rend plus difficile l'utilisation directe de méthodes de détection ou de suivi conçues pour des images perspectives.

Dans les travaux précédents du projet, un pipeline de détection de piétons basé sur plu-

sieurs projections gnomoniques a été mis en place. L'idée générale consiste à transformer l'image fisheye originale en plusieurs projections 2D proches d'images perspectives, puis à regrouper ces projections dans une image composite. Un détecteur YOLO pré-entraîné est ensuite appliqué sur cette image composite afin d'obtenir des boîtes candidates pour les piétons. Enfin, les détections produites sont reprojetees dans le repère de l'image fisheye. Cette stratégie permet de conserver l'intérêt du grand champ de vision tout en limitant l'effet des distorsions fisheye sur le détecteur.

Dans ce type de système, le module de suivi doit faire face à plusieurs difficultés qui influencent directement la stabilité des identités :

- l'apparence d'un même piéton peut évoluer au cours du temps, sous l'effet de la posture, de l'orientation, de l'éclairage, des vêtements ou d'accessoires ;
- les vecteurs d'apparence extraits à différents instants peuvent devenir instables, surtout en cas de changement rapide ou d'occlusion partielle ;
- plusieurs piétons peuvent présenter des apparences proches, ce qui rend l'association ambiguë lorsque l'association repose surtout sur la trame courante ;
- une mauvaise association peut contaminer la représentation d'une trajectoire et entraîner progressivement un ID switch.

Ainsi, la détection image par image ne suffit pas pour exploiter correctement une séquence vidéo. Un module de suivi est nécessaire pour relier les détections entre trames successives et maintenir un identifiant cohérent pour chaque piéton. C'est dans ce cadre que s'inscrit ce stage : améliorer la robustesse de cette association d'identité dans des vidéos fisheye, en particulier lorsque l'apparence des piétons varie au cours du temps.

3 Motivation et Objectifs

Dans le pipeline fisheye existant, le détecteur fournit des boîtes candidates pour les piétons, qui peuvent ensuite être utilisées par les étapes suivantes du système. Cependant, ces détections ne suffisent pas à elles seules pour garantir une analyse vidéo fiable. Dans un contexte de surveillance ou d'application embarquée, il est également nécessaire de conserver une identité cohérente pour chaque personne au fil des trames. Cette nécessité constitue la motivation principale du stage : améliorer la capacité du module de suivi à associer correctement les détections dans le temps, malgré les variations d'apparence.

Dans un premier temps, le travail s'appuie sur des méthodes de type suivi par détection, notamment DeepSORT et StrongSORT. Ces approches utilisent une prédiction de

mouvement et un vecteur d'apparence pour associer les détections aux trajectoires existantes. Elles constituent donc une base pertinente pour étudier le problème. Cependant, dans le contexte de ce stage, une représentation fondée principalement sur l'apparence d'une trame courante ou sur une mise à jour simple de l'historique ne suffit pas toujours à décrire une identité de manière stable. Lorsque la posture, l'angle de vue ou les vêtements des piétons changent progressivement, la représentation de la trajectoire doit pouvoir exploiter une information temporelle plus riche.

Nous distinguons donc deux niveaux d'historique : une mémoire à court terme, qui conserve les observations récentes pour suivre les changements immédiats, et une mémoire à long terme, qui garde des observations plus anciennes afin de maintenir une référence plus stable de l'identité.

L'objectif du stage est donc de concevoir et d'évaluer un module de suivi mieux adapté aux changements d'apparence. Plus précisément, le travail vise à :

- mettre en place une configuration de référence de suivi basée sur DeepSORT / StrongSORT et sur des caractéristiques OSNet ;
- analyser les situations dans lesquelles les changements d'apparence provoquent des erreurs d'association ou des changements d'identité ;
- concevoir des mécanismes de mémoire à court terme et à long terme afin de conserver des informations d'identité sur plusieurs échelles temporelles ;
- étudier l'utilisation de l'information temporelle et de mécanismes d'attention pour produire un score d'association plus robuste entre détections et trajectoires ;
- intégrer le module de suivi amélioré dans le pipeline fisheye / gnomonique existant ;
- évaluer l'effet des différents composants à travers des expériences et des études d'ablation.

4 Organisation du rapport

Ce rapport est organisé en cinq parties principales, qui suivent la progression du travail réalisé pendant le stage.

Introduction générale Cette partie présente le contexte du stage, le cadre général du projet, ainsi que les motivations et les objectifs du travail. Elle permet de situer le problème du suivi de piétons dans des vidéos fisheye et d'introduire la question de la stabilité des identités face aux changements d'apparence.

État de l’art Ce chapitre présente les travaux liés au sujet du stage. Il aborde d’abord les représentations d’apparence utilisées pour l’identification des personnes, puis les méthodes de suivi multi-objets telles que DeepSORT et MeMOT. Il introduit également plusieurs approches exploitant l’information temporelle et les mécanismes de mémoire.

Contribution Cette partie décrit les travaux réalisés pendant le stage. Elle présente la configuration de référence de suivi utilisée, les mécanismes de mémoire à court et long terme, ainsi que les pistes explorées pour améliorer l’association entre détections et trajectoires dans les situations de changement d’apparence.

Expérimentations et résultats Ce chapitre détaille les expérimentations menées pour évaluer les différentes approches. Il présente les données utilisées, les métriques d’évaluation, les comparaisons avec la configuration de référence et l’analyse des résultats obtenus.

Conclusion générale La dernière partie résume les principaux apports du stage, discute les limites observées pendant les expérimentations et propose plusieurs perspectives pour améliorer la robustesse du suivi dans de futurs travaux.

État de l’art

1 Introduction

Ce chapitre présente les travaux qui ont servi de base scientifique et technique à ce stage. Le problème étudié ne se limite pas à détecter des piétons dans une image fisheye, mais consiste aussi à conserver une identité stable lorsque leur apparence évolue au cours du temps. Pour cette raison, la revue bibliographique est organisée autour du paradigme du *suivi par détection*, qui associe les détections successives à l’aide d’informations de mouvement, d’apparence et d’historique de trajectoire.

L’état de l’art est donc organisé autour des méthodes utiles pour comprendre ce problème. Nous présentons d’abord les approches de suivi multi-objets fondées sur le *suivi par détection*, comme DeepSORT et StrongSORT. Ces méthodes constituent le cadre général de notre travail, car elles combinent une prédiction de mouvement, des détections image par image et des critères d’association entre trajectoires et observations courantes.

Une partie de la revue est aussi consacrée aux représentations d’apparence utilisées pour la ré-identification de personnes. Dans notre pipeline, OSNet est utilisé comme encodeur d’apparence de la configuration de référence, FastReID fournit les vecteurs d’apparence utilisés par le modèle temporel appris, puis AFLink complète le système par une liaison finale des fragments de trajectoire. Les autres approches de ré-identification robustes aux changements d’apparence permettent de comprendre les capacités et les limites des représentations extraites image par image.

Afin de garder cette revue centrée sur le sujet du stage, nous ne cherchons pas à présenter l’ensemble des méthodes de suivi multi-objets et de ré-identification. Les travaux retenus sont ceux qui éclairent directement les choix du projet : leur intérêt pour la cohérence des identités, leur capacité à prendre en compte les variations d’apparence entre trames successives, et leur compatibilité avec un pipeline fisheye/gnomonique fondé sur la détection puis le suivi.

Enfin, nous nous intéressons aux méthodes qui exploitent l'information temporelle et la mémoire des trajectoires. Cette partie est directement liée à la contribution du stage, qui consiste à enrichir une configuration de référence DeepSORT/StrongSORT avec une mémoire courte, une mémoire longue et un modèle appris pour évaluer l'association entre une détection courante et l'historique d'une trajectoire.

2 Travaux connexes

2.1 Suivi multi-objets par détection

DeepSORT DeepSORT [1] est l'une des méthodes de suivi multi-objets les plus directement liées à ce projet. Elle s'appuie sur le cadre *suivi par détection* de SORT [2] : le filtre de Kalman prédit d'abord la position des trajectoires existantes dans la trame courante, puis un coût d'association est construit entre ces prédictions et les détections disponibles. L'association est ensuite résolue par l'algorithme hongrois.

Contrairement à SORT, qui repose surtout sur le recouvrement spatial entre boîtes, DeepSORT ajoute un vecteur d'apparence de ré-identification pour décrire l'apparence de chaque détection. L'association ne dépend donc pas uniquement de la géométrie, mais aussi d'une distance d'apparence entre la détection courante et les trajectoires déjà suivies. DeepSORT introduit également une gestion explicite du cycle de vie des trajectoires, avec des états d'initialisation, de confirmation, de perte et de suppression, ainsi qu'un recours à l'IoU lorsque l'information d'apparence ne suffit pas. Ces mécanismes sont très proches de la structure du traqueur utilisé dans ce stage, ce qui fait de DeepSORT une base essentielle pour comprendre notre configuration de référence.

StrongSORT StrongSORT [3] prolonge DeepSORT en renforçant plusieurs composants du suivi, notamment la qualité du détecteur, l'encodeur ReID et la mise à jour des trajectoires. Il conserve une association fondée sur le mouvement et l'apparence, ce qui en fait une référence importante pour notre configuration de suivi.

Dans notre travail, cette logique DeepSORT/StrongSORT est reprise comme base : le mouvement reste modélisé par le filtre de Kalman, tandis que l'apparence est utilisée pour construire le coût d'association. Notre contribution ajoute ensuite une branche temporelle et un post-traitement AFLink.

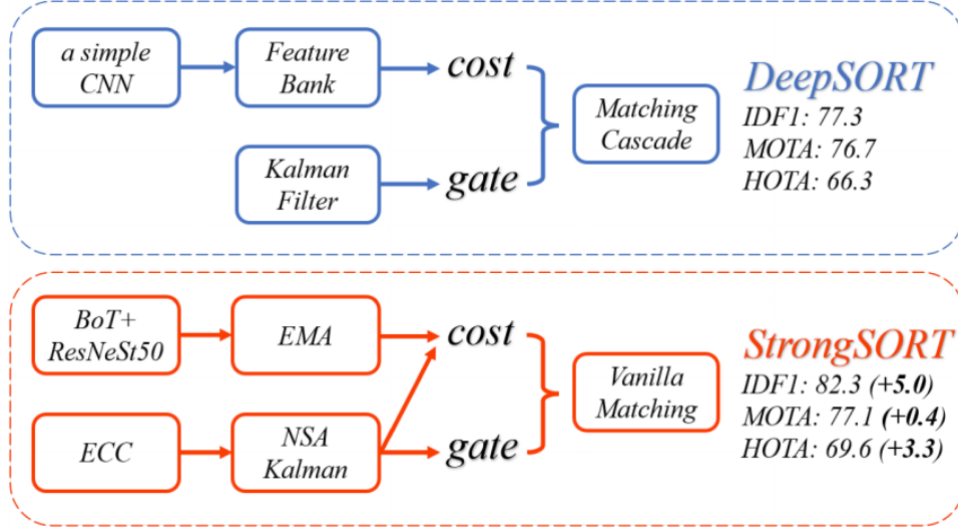


FIGURE 1 – Comparaison entre les structures DeepSORT et StrongSORT. La partie DeepSORT montre les deux branches principales utilisées dans notre configuration de référence : apparence et mouvement [3].

ByteTrack ByteTrack [4] est une autre méthode représentative du *suivi par détection*. Son idée principale est de ne pas éliminer trop tôt les boîtes de faible confiance, mais de les utiliser dans une association en deux étapes pour récupérer certains objets partiellement occultés ou sous-estimés par le détecteur. Notre travail n'utilise pas ByteTrack comme traqueur principal, mais cette méthode souligne un point important observé dans nos expériences : la qualité des détections, les faux négatifs et les choix de filtrage influencent directement les résultats de suivi.

BoT-SORT BoT-SORT [5] est un traqueur multi-objets qui combine une association fondée sur le mouvement avec des informations d'apparence issues d'un encodeur ReID. Il ajoute notamment des mécanismes comme la compensation du mouvement de caméra et une association plus robuste des trajectoires. Dans nos expériences, BoT-SORT est utilisé comme traqueur de référence externe, avec les mêmes détections que notre pipeline, afin de comparer la qualité d'association.

AFLink AFLink [3] est un module de post-traitement introduit dans StrongSORT++. Il ne remplace pas l'association trame par trame réalisée par l'algorithme hongrois, mais intervient après le suivi pour reconnecter des *fragments de trajectoire* séparés. Cette idée est liée à nos expériences récentes, car les occlusions ou les variations d'apparence peuvent

fragmenter une même identité en plusieurs trajectoires. AFLink peut donc améliorer la continuité des trajectoires, mais il reste un complément au traqueur principal plutôt qu’un substitut au modèle d’association.

3D Multi-Pedestrian ReID Le travail sur l’utilisation de la ré-identification dans le suivi 3D multi-piétons [?] étudie le rôle réel des informations d’apparence dans un traqueur en ligne. Les auteurs analysent différents encodeurs ReID, plusieurs stratégies de mémoire d’apparence, comme une galerie à fenêtre fixe, une moyenne cumulative et une moyenne exponentielle, ainsi que plusieurs façons de fusionner l’apparence avec les indices géométriques.

Cette étude montre que les caractéristiques ReID ne doivent pas nécessairement être intégrées de manière forte dans toutes les associations. Lorsqu’une information géométrique ou de mouvement est déjà fiable, une fusion trop directe peut introduire du bruit. En revanche, l’apparence devient utile comme information complémentaire, notamment pour récupérer des trajectoires non associées ou difficiles. Cette observation rejoint notre choix d’utiliser le score temporel et la récupération temporelle comme un appui lorsque le coût d’apparence de référence rejette certaines paires détection-trajectoire.

2.2 Représentation d’apparence pour la ré-identification

BoT / FastReID BoT [6] désigne une méthode de référence forte de ré-identification de personnes. Comme illustré dans la Figure 2, le modèle s’appuie généralement sur un réseau de base ResNet50 et utilise une *mise en commun moyenne globale* pour produire une représentation globale de l’apparence d’un piéton. BoT améliore cette méthode par plusieurs choix d’entraînement, notamment l’utilisation de *effacement aléatoire*, la modification du dernier pas du réseau pour conserver davantage d’information spatiale, et l’ajout de BNNeck.

BNNeck permet de séparer les caractéristiques utilisées pour apprendre une distance entre identités de celles utilisées pour la classification d’identité. Le modèle peut ainsi apprendre à rapprocher les observations d’une même personne tout en conservant une bonne capacité de classification. Lors de l’inférence, la représentation produite sert de vecteur d’apparence pour comparer les personnes détectées.

FastReID [7] est un cadre PyTorch modulaire pour l’entraînement et l’inférence de modèles de ré-identification. Il fournit les composants nécessaires au prétraitement des images, au choix du réseau de base, à l’agrégation des caractéristiques, aux fonctions

de perte et à l'extraction des vecteurs d'apparence. Dans notre pipeline, les vecteurs d'apparence utilisés par la branche temporelle et par le TemporalAttentionScorer sont extraits avec FastReID.

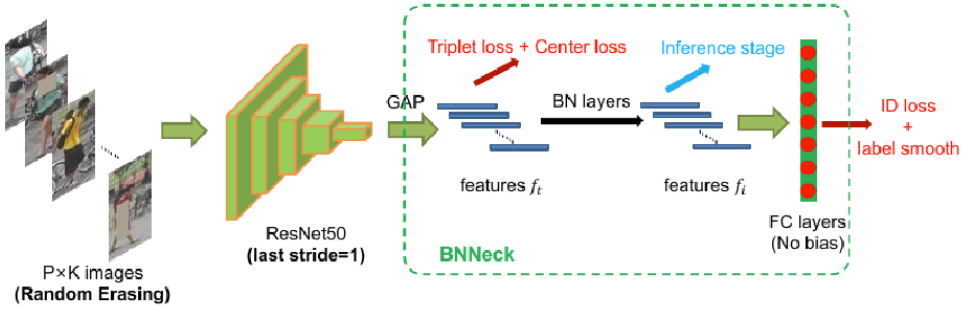


FIGURE 2 – Chaînes de traitement de la méthode standard et de la méthode BoT modifiée avec astuces d'entraînement et BNNeck [6].

OSNet OSNet [8] propose une architecture de ré-identification fondée sur l'apprentissage de caractéristiques à plusieurs échelles. Le réseau utilise plusieurs branches avec des champs réceptifs différents afin de capturer à la fois la silhouette globale et des détails plus locaux. Un mécanisme d'agrégation dynamique combine ensuite ces informations pour produire un vecteur d'apparence plus discriminant.

OSNet est utilisé dans notre implémentation comme encodeur d'apparence de la configuration de référence. Dans un contexte fisheye, où l'échelle et la forme apparente des personnes peuvent varier fortement selon leur position dans l'image, sa représentation multi-échelle est pertinente pour construire le coût d'apparence de base.

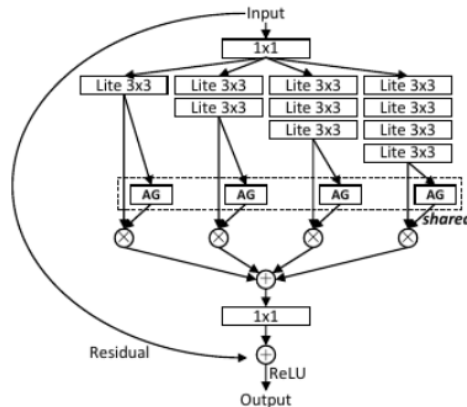


FIGURE 3 – Bloc goulot de référence et bloc goulot OSNet proposé avec porte d'agrégation [8].

FeatureSORT FeatureSORT [?] complète les représentations ReID classiques avec des attributs visuels plus explicites. La méthode ajoute au détecteur des sorties liées à la couleur des vêtements, au style vestimentaire et à la direction du corps ou du mouvement, puis utilise ces informations dans le coût d’association avec le mouvement et les caractéristiques ReID.

Cette approche montre qu’un seul vecteur d’apparence global ne suffit pas toujours pour distinguer des personnes proches ou visuellement ambiguës. Des attributs plus interprétables peuvent fournir des indices supplémentaires pour l’association. Il constitue une perspective intéressante pour notre travail FiTAC-MOT : Il peut ajouter au modèle temporel des attributs explicites tels que la couleur des vêtements, les accessoires ou l’orientation de la posture.

2.3 Modélisation de l’identité par mémoire

MeMOT MeMOT [9] est le travail le plus important pour situer notre mécanisme de mémoire. Son idée principale est que, dans le suivi multi-objets, l’identité d’un objet ne doit pas être représentée uniquement par la trame courante ou par la dernière observation. Elle doit aussi s’appuyer sur les informations mémorisées dans les trames précédentes.

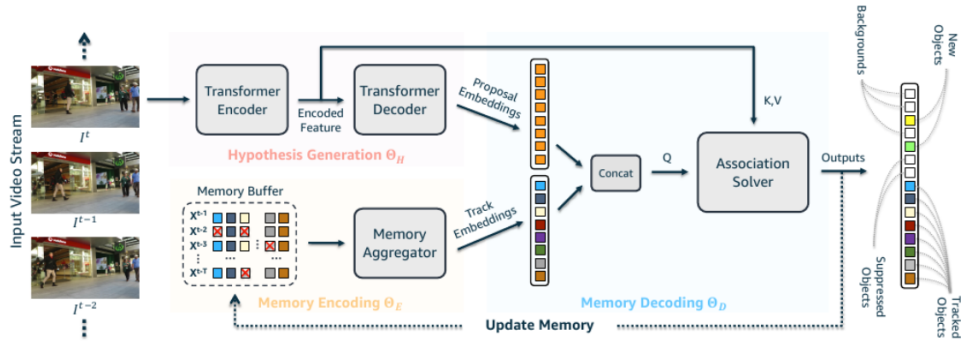


FIGURE 4 – Architecture globale de MeMOT avec génération d’hypothèses, encodage mémoire et modules de décodage [9].

Comme montré dans la Figure 4, MeMOT comporte trois modules principaux. Le module de génération d’hypothèses produit des candidats pour la trame courante. Le module d’encodage de mémoire lit les informations historiques des objets déjà suivis dans un *tampon mémoire* et les agrège sous forme de *représentations de trajectoires*. Enfin, le module de décodage combine les candidats courants et les représentations historiques afin de résoudre simultanément la détection et l’association, en distinguant les nouveaux

objets, les objets déjà suivis et les régions de fond.

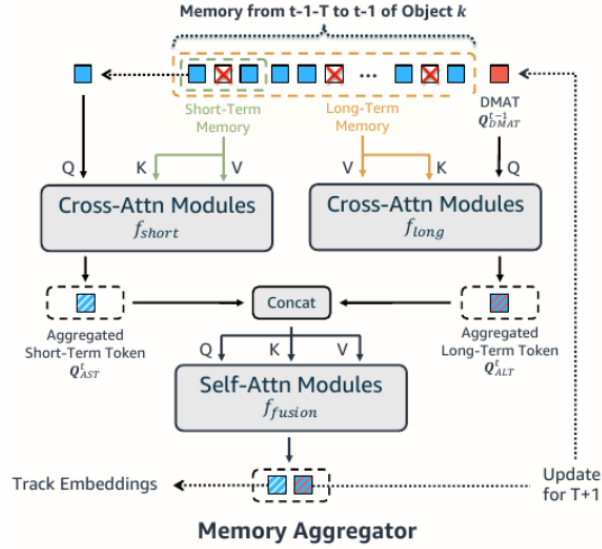


FIGURE 5 – Agrégation de mémoire dans MeMOT avec modules d’attention à court terme, à long terme et globale [9].

Le second schéma détaille le module d’agrégation de mémoire. MeMOT distingue une mémoire à court terme, qui décrit les observations récentes d’un objet, et une mémoire à long terme, qui conserve des informations plus anciennes et plus stables. Les deux mémoires sont agrégées par des mécanismes d’attention, puis fusionnées pour produire une représentation de trajectoire utilisée dans l’association.

MeMOT produit une représentation de trajectoire plus stable à partir de la mémoire, tandis que notre travail adopte une version plus légère dans un pipeline DeepSORT/StrongSORT. Nous ne reproduisons pas l’architecture Transformer complète de MeMOT ; nous reprenons plutôt l’idée d’utiliser l’historique pour renforcer l’association d’identité au niveau des caractéristiques ReID. Dans notre système, cette idée se traduit par un modèle qui lit la caractéristique de la détection courante, la mémoire courte et la mémoire longue, puis apprend la compatibilité entre cette détection et l’historique de la trajectoire.

History-Aware MOT History-Aware MOT [?] s’intéresse aussi à l’utilisation de l’historique des trajectoires pour améliorer les caractéristiques ReID dans le suivi multi-objets. Les auteurs partent du constat que l’espace ReID général n’est pas toujours adapté à une séquence donnée, car le suivi doit surtout distinguer quelques personnes visuellement

proches présentes dans la même vidéo. La Figure 6 illustre ce principe : les caractéristiques historiques des trajectoires et les caractéristiques des détections courantes sont utilisées pour construire un espace plus adapté à l’association dans la séquence.

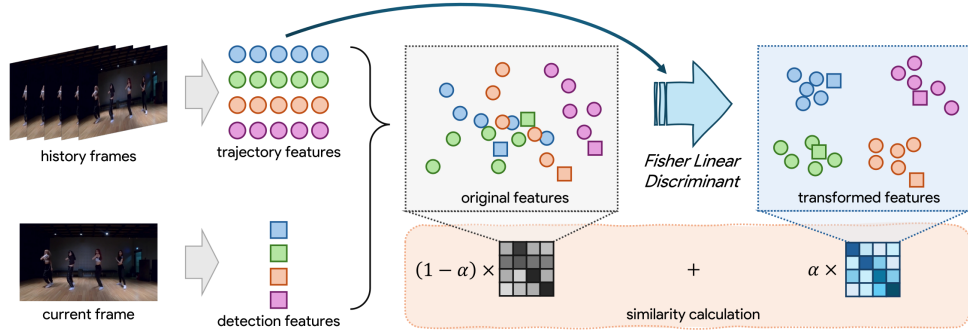


FIGURE 6 – Principe général de History-Aware MOT : les caractéristiques historiques des trajectoires sont utilisées pour adapter l’espace ReID à la séquence courante [?].

Comme le montre la Figure 6, les trames historiques fournissent les caractéristiques des trajectoires, tandis que la trame courante fournit les caractéristiques des détections à associer. La méthode considère ensuite les caractéristiques historiques de chaque trajectoire comme une classe, puis calcule une projection fondée sur le discriminant linéaire de Fisher. L’objectif est d’augmenter la séparation entre les trajectoires différentes et de réduire la variance des caractéristiques appartenant à une même trajectoire. Les caractéristiques projetées sont enfin combinées avec l’espace ReID initial pour calculer une similarité plus adaptée à la séquence courante.

History-Aware MOT introduit également un centre de trajectoire pondéré dans le temps. Les observations récentes reçoivent un poids plus important que les observations anciennes. Cette idée est proche de notre motivation : l’historique d’une trajectoire doit être exploité avec une attention particulière à son évolution temporelle. Elle confirme l’intérêt d’utiliser l’historique ReID pour rendre l’association plus robuste.

3 Conclusion

Les travaux présentés dans ce chapitre montrent différentes manières d’aborder le problème de l’association d’identité dans le suivi multi-objets. DeepSORT et StrongSORT fournissent le cadre de base du *suivi par détection*, dans lequel les détections sont reliées au cours du temps à partir d’informations de mouvement et d’apparence. OSNet est retenu comme encodeur d’apparence de la configuration de référence, tandis que FastReID est utilisé pour alimenter le modèle temporel appris. OSNet, FastReID et les approches de ré-identification montrent plus généralement que la qualité du vecteur d’apparence joue un rôle central dans la stabilité des identités, même si ces représentations restent sensibles aux variations visuelles.

Les méthodes exploitant l’information temporelle, comme les représentations de fragments de trajectoire, ainsi que les approches basées sur la mémoire, comme MeMOT, indiquent que l’apparence extraite dans une seule image ne suffit pas toujours à décrire une identité de manière stable. L’historique d’une trajectoire et les relations temporelles entre observations peuvent fournir des informations utiles pour l’association. Ces constats motivent le travail présenté dans les chapitres suivants : partir d’une configuration de référence DeepSORT/StrongSORT avec OSNet comme encodeur d’apparence, puis introduire une mémoire de trajectoire et un modèle appris d’association temporelle fondé sur FastReID afin d’améliorer la stabilité des identités dans des séquences fisheye/gnomoniques.

Contribution

1 Introduction

Ce chapitre présente les contributions réalisées pendant le stage. Le travail s’inscrit dans la continuité du pipeline de détection fisheye déjà existant : les piétons sont d’abord détectés à partir de projections gnomoniques, puis les boîtes obtenues sont reprojetées dans l’image fisheye. La contribution principale se concentre donc sur l’étape suivante : associer les détections dans le temps afin de conserver une identité stable pour chaque piéton.

Comme présenté dans l’état de l’art, les méthodes de suivi de type DeepSORT ou StrongSORT utilisent fortement des vecteurs d’apparence extraits image par image. Cette représentation est efficace lorsque l’apparence d’un piéton reste relativement stable. Elle devient cependant plus fragile lorsque la posture, l’orientation du corps, une occlusion partielle ou la déformation liée à la caméra fisheye modifient progressivement les caractéristiques extraites. Dans ce cas, comparer uniquement la détection courante avec une représentation récente de la trajectoire peut conduire à des associations incorrectes et à des changements d’identifiant.

L’objectif de cette contribution est donc de renforcer la représentation d’identité utilisée par le traqueur. Nous nommons le système proposé FiTAC-MOT, pour *Fisheye Temporal Appearance-Change Multi-Object Tracking*. FiTAC-MOT part d’une configuration de référence de suivi inspirée de DeepSORT, avec OSNet comme encodeur d’apparence principal. Cette configuration de référence calcule un coût d’apparence C^{base} à partir des vecteurs d’apparence OSNet. En parallèle, une branche temporelle utilise des vecteurs d’apparence extraits avec FastReID pour produire un coût temporel appris C^{temp} . La dernière étape du système applique AFLink comme post-traitement afin de relier certains fragments de trajectoire.

Dans un second temps, nous étudions un score d’association temporel appris. Ce mo-

dule utilise la caractéristique FastReID de la détection courante ainsi que l'historique FastReID de la trajectoire pour estimer si une association est cohérente. Il intervient à plusieurs niveaux de la matrice de coût : fusion avec le coût d'apparence, correction du filtrage géométrique dans certains cas temporellement cohérents, puis règle de secours avant l'application du seuil d'association.

Enfin, le chapitre décrit l'intégration de ces modules dans le pipeline fisheye/gnomonique. Une attention particulière est portée à la conversion entre les boîtes issues de la détection, les régions utilisées pour l'extraction des vecteurs d'apparence ReID, les boîtes utilisées par le traqueur et les trajectoires finales traitées par AFLink.

2 Configuration de référence de type DeepSORT avec vecteurs d'apparence OSNet

2.1 Principe suivi par détection de la configuration de référence

La configuration de référence utilisée dans ce travail suit un cadre de type *suivi par détection*. Le système construit les trajectoires à partir des sorties d'un module de détection, qui fournit pour chaque trame des boîtes candidates de piétons. Ces détections sont ensuite associées aux trajectoires déjà existantes.

Dans notre cas, cette étape intervient après le pipeline de détection fisheye. L'image fisheye est d'abord transformée par projections gnomoniques, puis un détecteur produit des boîtes de piétons qui sont reprojettées dans le repère fisheye. Le module de suivi reçoit ensuite ces boîtes, extrait les caractéristiques d'apparence nécessaires et établit les associations entre trames successives.

Le détecteur fournit les candidats visuels. Le traqueur se concentre sur la cohérence temporelle des identités. Dans le code, un module d'adaptation relie ces deux parties : les boîtes issues du pipeline fisheye sont converties dans un format utilisable par le traqueur, puis elles entrent dans les étapes d'extraction ReID et d'association.

Le pipeline proposé peut être résumé de la manière suivante pour chaque trame. La Figure 1 donne une vue globale de cette organisation :

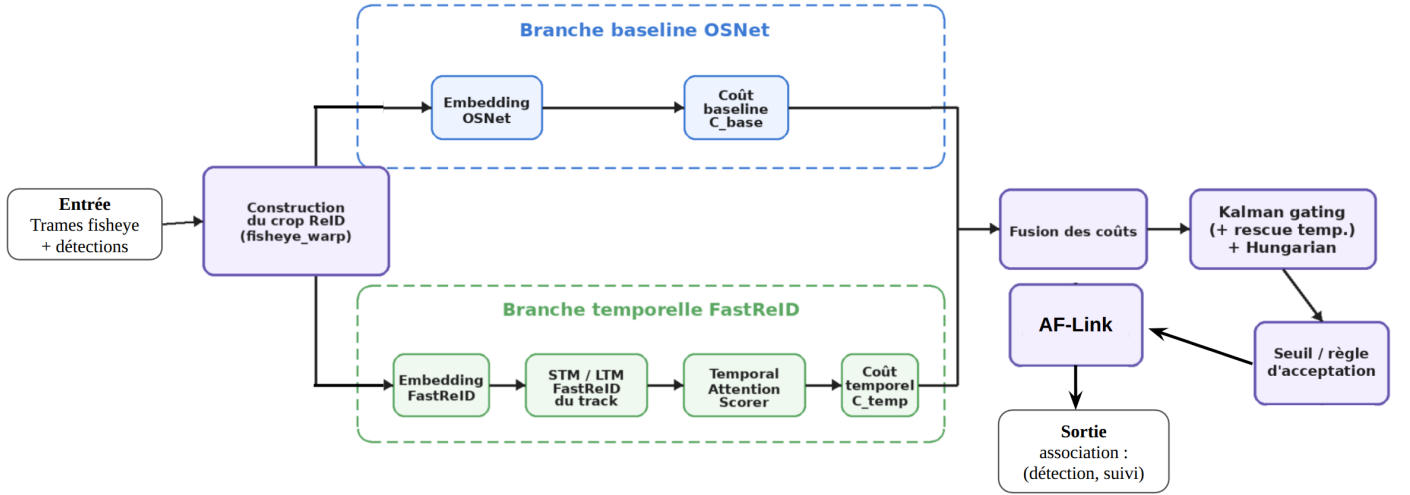


FIGURE 1 – Structure globale simplifiée de FiTAC-MOT avec configuration de référence OSNet, correction temporelle FastReID et post-traitement AFLink.

La figure montre le rôle du traqueur dans la chaîne complète. Après la construction du recadrage pour les encodeurs, le flux se sépare en deux branches : la branche OSNet calcule le coût de référence C^{base} , tandis que la branche FastReID produit le coût temporel C^{temp} . Les deux coûts sont ensuite fusionnés avant le filtrage géométrique de Kalman, l’association hongroise et la règle d’acceptation.

2.2 Extraction des vecteurs d’apparence : OSNet pour la configuration de référence et FastReID pour le modèle temporel

La Figure 2 illustre la différence entre la boîte utilisée par le traqueur et la région envoyée à l’encodeur ReID. Dans le pipeline fisheye/gnomonique, les boîtes utilisées pour la détection, pour le suivi et pour l’extraction ReID ne jouent pas exactement le même rôle. La détection produit des boîtes orientées dans l’image fisheye, souvent plus proches de l’orientation réelle du piéton. En revanche, le traqueur maintient une représentation horizontale de type *tlwh*, compatible avec les mécanismes classiques de DeepSORT, notamment le filtre de Kalman et le *filtrage de mouvement*. Cette représentation est utile pour modéliser la position et la taille de la trajectoire, mais elle n’est pas toujours la meilleure région visuelle pour extraire une caractéristique d’apparence.

L’encodeur ReID reçoit en entrée une région image rectangulaire. Dans notre implémentation, cette région extraite est redimensionnée en 256×128 , ce qui correspond au format couramment utilisé dans les méthodes de référence de ré-identification de per-

sonnes. Il faut donc construire, à partir de la détection fisheye, une région visuelle adaptée à l'encodeur d'apparence.

Une première possibilité consiste à utiliser un recadrage fisheye aligné sur les axes. On calcule alors un rectangle horizontal englobant la boîte orientée, puis on découpe directement cette région dans l'image fisheye. Cette solution est simple, mais elle ajoute souvent beaucoup de fond autour du piéton. Dans une scène réelle, ce fond peut contenir des murs, des objets proches ou d'autres personnes. Ces éléments entrent alors dans l'encodeur ReID et peuvent rendre le vecteur d'apparence moins stable.

Pour cette raison, nous avons étudié le mode `fisheye_warp`. Dans ce cas, le système utilise les quatre coins de la boîte orientée et applique une transformation affine pour redresser la région correspondante. On obtient ainsi une région plus compacte, mieux alignée avec le corps du piéton et contenant moins de bruit de fond qu'un simple rectangle englobant. La région ReID extraite est ensuite utilisée par les deux branches ReID : OSNet pour la configuration de référence et FastReID pour le modèle temporel.



(a) Représentation `fisheye_warp` : boîte orientée puis région ReID redressée.

(b) Représentation alignée sur les axes : boîte horizontale puis recadrage rectangulaire.

FIGURE 2 – Différence entre la représentation `fisheye_warp` utilisée pour l'extraction ReID et la représentation alignée sur les axes utilisée par le traqueur. La sous-figure (a) montre le redressement de la région à partir de la boîte orientée, tandis que la sous-figure (b) montre le recadrage horizontal qui peut contenir davantage de fond.

La boîte de suivi sert à maintenir l'état géométrique de la trajectoire, tandis que la région `fisheye_warp` sert à produire une caractéristique d'apparence plus propre.

Comme présenté dans l'état de l'art, OSNet fournit une représentation d'apparence multi-échelle, adaptée aux variations de taille et de forme observées dans les images fisheye. Dans notre pipeline, il est utilisé comme encodeur d'apparence de la branche de référence avec les poids `osnet_x1_0_imagenet.pth`. Il produit, pour chaque détection d_t , un vecteur d'apparence :

$$f_t^{OSNet} = E_{OSNet}(I(d_t)). \quad (1)$$

La branche temporelle utilise FastReID avec sa configuration et ses poids dédiés pour produire une seconde représentation :

$$f_t^{FastReID} = E_{FastReID}(I(d_t)). \quad (2)$$

Dans ces notations, $I(d_t)$ désigne la région image extraite à partir de la détection d_t . Le vecteur d'apparence OSNet sert à construire le coût d'apparence de la configuration de référence, tandis que le vecteur d'apparence FastReID alimente les mémoires STM/LTM (mémoire courte / mémoire longue) du modèle temporel et le TemporalAttentionScorer.

Dans une trame, les vecteurs d'apparence des N détections valides sont organisés en matrices de caractéristiques, puis normalisés par la norme L_2 :

$$\hat{f}_t = \frac{f_t}{\|f_t\|_2}. \quad (3)$$

Après cette normalisation, la comparaison entre deux vecteurs d'apparence repose surtout sur leur direction dans l'espace de caractéristiques, ce qui est cohérent avec l'utilisation de distances cosinus. Ainsi, chaque détection transmise au traqueur contient une boîte horizontale $tlwh$, utilisée par le filtre de Kalman et les contraintes géométriques, un vecteur d'apparence OSNet pour le coût de référence, et un vecteur d'apparence FastReID pour le coût temporel appris.

2.3 Association détection-trajectoire : Kalman, coût d'apparence et association hongroise

Après l'extraction des vecteurs d'apparence de la trame courante, la configuration de référence doit associer les détections aux trajectoires existantes. La logique reste proche de DeepSORT. Chaque trajectoire est d'abord prédite dans la trame courante par un filtre de Kalman. Cette prédiction fournit une estimation géométrique de la position attendue du piéton.

Ensuite, le système calcule un coût de référence entre les détections et les trajectoires à partir des vecteurs d'apparence OSNet. Dans cette branche, OSNet joue le rôle d'encodeur

d'apparence de référence. Le coût C^{base} repose sur une distance cosinus entre le vecteur d'apparence OSNet de la détection courante et les vecteurs d'apparence OSNet associés à la trajectoire, puis il est utilisé comme coût principal avant la correction temporelle apprise. Un coût faible indique donc une forte similarité visuelle entre la détection et la trajectoire. Après la fusion avec le coût temporel, l'algorithme hongrois utilise le coût fusionné C^{fused} pour résoudre l'affectation :

$$\min_{\pi} \sum_i C_{i,\pi(i)}^{fused}. \quad (4)$$

Ici, π désigne l'affectation choisie par l'association. Le *filtrage géométrique* de Kalman intervient comme une contrainte géométrique : les associations trop éloignées de la position prédite sont ignorées ou reçoivent un coût très élevé.

L'organisation de l'association reste celle d'un traqueur de type DeepSORT. Les trajectoires confirmées sont associées à partir du coût fusionné, qui combine le coût de référence OSNet et le coût temporel FastReID. Cette combinaison permet de conserver la cohérence géométrique du traqueur tout en ajoutant une correction temporelle lorsque l'apparence évolue. La section suivante détaille le score temporel appris utilisé pour C^{temp} .

3 TemporalAttentionScorer avec vecteurs d'apparence FastReID

Le coût d'apparence OSNet fournit une base utile pour l'association, mais il reste défini à partir de distances entre vecteurs d'apparence. Or, dans certaines situations, l'observation récente d'une trajectoire est plus informative, par exemple après un changement de posture. Dans d'autres cas, l'historique plus long peut fournir un repère plus fiable, surtout lorsque les observations récentes sont perturbées.

Pour prendre en compte cette différence, nous introduisons un score temporel appris. Le modèle fournit un coût supplémentaire pour chaque paire candidate détection-trajectoire. Ce coût est calculé à partir de la caractéristique FastReID courante, de la mémoire courte FastReID et de la mémoire longue FastReID de la trajectoire. Il indique si l'association semble cohérente du point de vue temporel et visuel.

3.1 Vue d'ensemble du TemporalAttentionScorer

Le *TemporalAttentionScorer* est un modèle léger ajouté au niveau de l'association. Son unité d'entrée n'est pas toute la trame, mais une paire candidate :

$$(f_t, T_i), \quad (5)$$

où f_t est la caractéristique FastReID de la détection courante et T_i une trajectoire existante. Pour cette trajectoire, le modèle lit aussi la mémoire courte M_s^i et la mémoire longue M_l^i . L'entrée réelle peut donc s'écrire :

$$(f_t, M_s^i, M_l^i). \quad (6)$$

La mémoire courte correspond aux observations récentes de la trajectoire. Sa longueur maximale est de 5 et les caractéristiques sont organisées de la plus récente à la plus ancienne. La mémoire longue couvre une période plus étendue, avec une longueur maximale de 30. Pour les trajectoires récentes, certaines positions peuvent être vides. Le modèle conserve alors les longueurs valides et utilise des masques pour que les positions de remplissage ne soient pas interprétées comme de vraies observations.

Concrètement, ces deux mémoires sont des tampons de caractéristiques FastReID associés à chaque trajectoire. La mémoire courte M_s^i est une séquence contenant les derniers vecteurs d'apparence FastReID de la trajectoire i . La mémoire longue M_l^i est une autre séquence, plus longue, qui conserve des vecteurs d'apparence plus anciens de la même trajectoire. Elles peuvent être vues comme des tableaux de taille fixe :

$$M_s^i \in \mathbb{R}^{T_s \times D}, \quad M_l^i \in \mathbb{R}^{T_l \times D}, \quad (7)$$

où D est la dimension du vecteur d'apparence FastReID, $T_s \leq 5$ et $T_l \leq 30$. Lorsqu'une trajectoire est mise à jour, la nouvelle caractéristique FastReID est ajoutée dans ces tampons selon la règle de construction de l'historique temporel. Le TemporalAttentionScorer lit ensuite ces séquences comme des jetons temporels.

Le modèle produit un score scalaire :

$$s_{i,t} = F_\theta(f_t, M_s^i, M_l^i). \quad (8)$$

La fonction F_θ correspond au TemporalAttentionScorer, dont θ regroupe les paramètres appris.

Après application d'une fonction sigmoïde, ce score peut être interprété comme la probabilité que la détection appartienne à la trajectoire. L'idée est inspirée de MeMOT,

dans le sens où l'identité d'un objet est représentée à partir d'une mémoire historique. Dans notre cas, le modèle travaille au niveau des caractéristiques ReID et sert à évaluer une paire candidate, plutôt qu'à réaliser une détection et un suivi de bout en bout.

3.2 Architecture du TemporalAttentionScorer

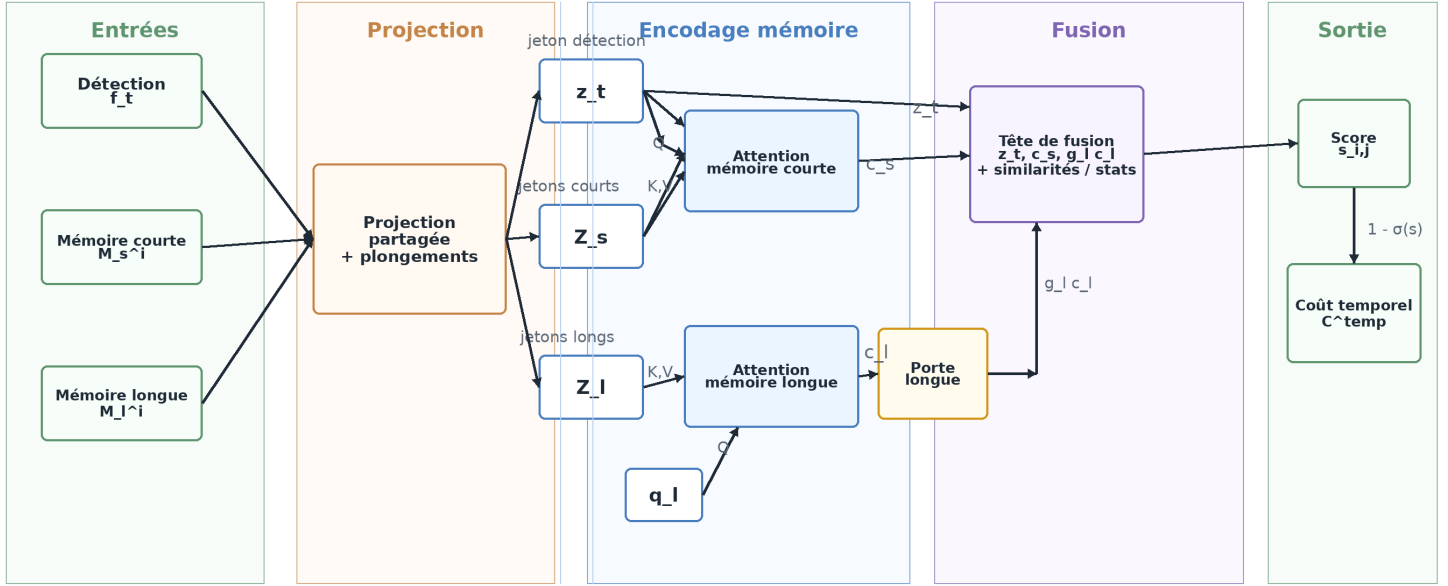


FIGURE 3 – Architecture compacte du TemporalAttentionScorer. Le modèle reçoit la caractéristique FastReID courante, la mémoire courte et la mémoire longue de la trajectoire, puis produit le coût temporel C^{temp} utilisé dans l'association.

Comme le montre la Figure 3, le modèle reçoit trois entrées : la caractéristique FastReID de la détection courante, la mémoire courte FastReID de la trajectoire et sa mémoire longue FastReID. Ces trois entrées sont d'abord envoyées dans une projection partagée, puis complétées par des vecteurs de position, de temps et de type. On note :

$$z_t = W f_t, \quad Z_s = W M_s, \quad Z_l = W M_l, \quad (9)$$

où W est la projection apprise vers l'espace caché, z_t le jeton de la détection, Z_s les jetons de mémoire courte et Z_l les jetons de mémoire longue.

L'architecture se décompose ensuite en cinq étapes : une projection partagée des entrées, une branche d'encodage de la mémoire courte, une branche d'encodage de la mémoire longue, un mécanisme de *porte longue*, puis une tête de fusion et de scoring. Les formules détaillées de ces différents blocs sont précisées dans les sous-parties suivantes.

Représentation temporelle de la trajectoire. Pour qu'un score temporel soit possible, une trajectoire doit conserver son historique d'apparence. Dans notre implémentation, chaque nouvelle détection associée à une trajectoire fournit une caractéristique ReID qui est ajoutée à cet historique. On peut écrire :

$$T_i = \{f_i^{t_1}, f_i^{t_2}, \dots, f_i^{t_n}\}. \quad (10)$$

Ces caractéristiques sont normalisées avant les comparaisons :

$$\hat{f} = \frac{f}{\|f\|_2}. \quad (11)$$

L'historique est ensuite séparé en mémoire courte et mémoire longue. La mémoire courte décrit l'état récent de l'apparence. Elle est utile lorsque le piéton change rapidement de posture ou d'orientation. La mémoire longue conserve une information plus large sur l'identité. Elle peut aider lorsque les observations récentes sont instables. Les trajectoires n'ont pas tous la même longueur d'historique. Une trajectoire récente peut ne contenir que quelques observations. Pour garder un format fixe, le système utilise du remplissage et conserve les longueurs valides. Les masques empêchent le modèle d'utiliser les positions vides comme si elles étaient de vraies caractéristiques.

Encodage de la mémoire courte. Dans les équations suivantes, MultiTteAttn désigne l'attention multi-têtes standard. Pour une requête Q , des clés K et des valeurs V , elle est définie par :

$$\text{MultiTteAttn}(Q, K, V) = \text{Concat}(tte_1, \dots, tte_h)W^O, \quad (12)$$

avec

$$tte_r = \text{softmax}\left(\frac{QW_r^Q(KW_r^K)^T}{\sqrt{d_h}}\right)VW_r^V. \quad (13)$$

Ici, h est le nombre de têtes d'attention, d_h la dimension d'une tête, et W_r^Q , W_r^K , W_r^V , W^O sont des matrices apprises.

La branche de mémoire courte sélectionne les observations récentes les plus utiles pour la détection courante. Après projection, la mémoire courte est notée :

$$Z_s = [z_s^1, z_s^2, \dots, z_s^{T_s}], \quad Z_s \in \mathbb{R}^{T_s \times H}. \quad (14)$$

Dans l'implémentation actuelle, T_s vaut au plus 5, et H désigne la dimension cachée après projection.

La détection courante est utilisée comme requête, tandis que les caractéristiques de la mémoire courte sont utilisées comme clés et valeurs :

$$c_s = \text{MultiTteAttn}(Q = z_t, K = Z_s, V = Z_s). \quad (15)$$

Le vecteur c_s est donc un résumé court terme construit à partir des observations récentes. L'intérêt de l'attention est de ne pas supposer que la dernière trame est toujours la plus fiable. Une détection récente peut être floue, mal recadrée ou perturbée par le fond. Le modèle peut alors se référer à une autre observation proche dans le temps.

Encodage de la mémoire longue. La branche longue cherche à extraire une représentation plus globale de la trajectoire. La mémoire longue projetée est notée :

$$Z_l = [z_l^1, z_l^2, \dots, z_l^{T_l}], \quad Z_l \in \mathbb{R}^{T_l \times H}. \quad (16)$$

Sa longueur maximale est plus grande que celle de la mémoire courte. Comme précédemment, H correspond à la dimension cachée.

Cette branche utilise un jeton de requête apprenable q_l . Ce jeton est défini comme un paramètre du modèle :

$$q_l = \theta_q, \quad \theta_q \in \mathbb{R}^{1 \times H}. \quad (17)$$

Il est initialisé avec les paramètres du TemporalAttentionScorer, puis optimisé pendant l'entraînement par rétropropagation avec les autres poids du modèle. Il sert de requête globale pour agréger les jetons de mémoire longue :

$$c_l = \text{MultiTteAttn}(Q = q_l, K = Z_l, V = Z_l). \quad (18)$$

Le vecteur c_l correspond à un contexte d'identité extrait de l'historique long. Comme le montre la Figure 4, cette information passe par un mécanisme de porte. L'historique peut contenir des observations anciennes, parfois moins compatibles avec la détection courante. Le modèle calcule donc une porte long terme :

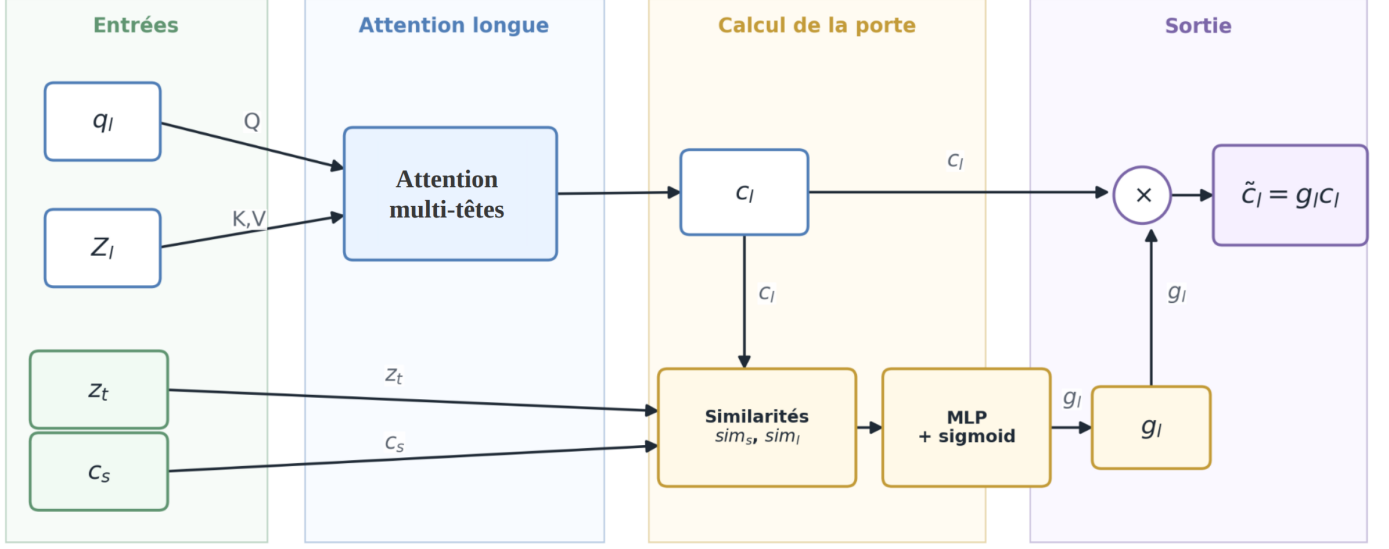


FIGURE 4 – Module attention de mémoire longue et mécanisme de porte longue.

Le module présenté dans la figure compare donc la détection courante avec les contextes court et long, puis produit un poids qui contrôle l'influence de la mémoire longue.

$$\text{sim}_s = \langle \hat{z}_t, \hat{c}_s \rangle, \quad \text{sim}_l = \langle \hat{z}_t, \hat{c}_l \rangle, \quad (19)$$

$$g_l = \sigma(\text{MLP}([z_t, c_s, c_l, \text{sim}_s, \text{sim}_l])), \quad (20)$$

et utilise

$$\tilde{c}_l = g_l c_l \quad (21)$$

dans la fusion finale. Ce mécanisme permet de réduire l'influence de la mémoire longue lorsqu'elle n'est pas compatible avec l'apparence actuelle.

Fusion et calcul du score d'association. Après l'encodage des deux mémoires, le modèle doit produire un score scalaire pour la paire (d_t, T_i) . La Figure 5 résume cette étape de fusion. La fusion reçoit trois représentations principales : la détection courante z_t , le contexte court c_s et le contexte long pondéré $\tilde{c}_l$.

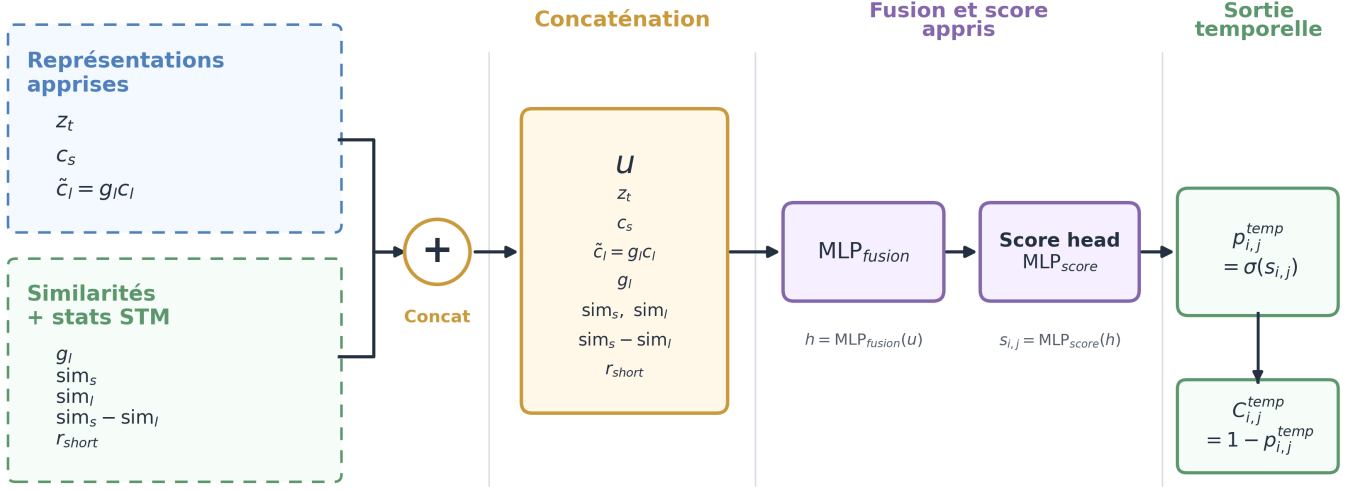


FIGURE 5 – Fusion des représentations et calcul du score d’association.

Dans cette tête de score, l’entrée de fusion regroupe les représentations apprises et les informations explicites utilisées pour comparer la détection avec les mémoires. La partie gauche de la figure distingue deux groupes d’entrées : les représentations apprises z_t, c_s et $\tilde{c}_l$, puis les informations explicites $g_l, \text{sim}_s, \text{sim}_l, \text{sim}_s - \text{sim}_l$ et r_{short} .

Le modèle calcule par exemple les similarités :

$$\text{sim}_s = \langle \hat{z}_t, \hat{c}_s \rangle, \quad \text{sim}_l = \langle \hat{z}_t, \hat{c}_l \rangle. \quad (22)$$

Il utilise également $\text{sim}_s - \text{sim}_l$, la valeur de la porte longue, et des statistiques de similarité entre la détection et les caractéristiques de la mémoire courte. Ces statistiques, regroupées dans r_{short} , comprennent la similarité avec la dernière observation, la moyenne, le maximum et le minimum. Si $Z_s = \{z_s^1, \dots, z_s^{T_s}\}$, on peut les écrire sous la forme :

$$r_{\text{short}} = [\langle \hat{z}_t, \hat{z}_s^1 \rangle, \text{mean}_k \langle \hat{z}_t, \hat{z}_s^k \rangle, \max_k \langle \hat{z}_t, \hat{z}_s^k \rangle, \min_k \langle \hat{z}_t, \hat{z}_s^k \rangle]. \quad (23)$$

Ces valeurs aident le modèle à vérifier si la détection courante reste cohérente avec l’historique court terme de la trajectoire.

L’opération de concaténation représentée par le symbole $+$ dans la figure construit le vecteur d’entrée :

$$u = [z_t, c_s, \tilde{c}_l, g_l, \text{sim}_s, \text{sim}_l, \text{sim}_s - \text{sim}_l, r_{\text{short}}]. \quad (24)$$

Le MLP de fusion transforme ce vecteur en une représentation intermédiaire :

$$h = \text{MLP}_{\text{fusion}}(u). \quad (25)$$

Le bloc *tête de score* correspond ensuite au MLP de scoring. Son rôle est simplement de projeter h vers un logit scalaire pour la paire trajectoire-détection :

$$s_{i,j} = \text{MLP}_{score}(h). \quad (26)$$

Ce logit est transformé en probabilité temporelle, puis en coût temporel :

$$p_{i,j}^{temp} = \sigma(s_{i,j}), \quad C_{i,j}^{temp} = 1 - p_{i,j}^{temp}. \quad (27)$$

Ainsi, plus $p_{i,j}^{temp}$ est élevé, plus l'association est considérée comme cohérente. À l'inverse, un faible $C_{i,j}^{temp}$ indique une association temporellement plausible.

3.3 Apprentissage supervisé du score temporel

Le TemporalAttentionScorer est entraîné de manière supervisée sur des paires détection-trajectoire. Chaque échantillon contient la caractéristique FastReID de la détection courante, les mémoires FastReID de la trajectoire candidate et une étiquette binaire :

$$x = (f_t, M_s^i, M_l^i), \quad y \in \{0, 1\}. \quad (28)$$

La valeur $y = 1$ signifie que la détection et la trajectoire correspondent à la même identité. La valeur $y = 0$ indique qu'ils appartiennent à des identités différentes.

Les paires d'entraînement sont construites à partir des propositions réellement produites par le pipeline. Ce choix rapproche l'apprentissage des conditions d'inférence. En pratique, une détection peut être proche de plusieurs trajectoires, et certains candidats négatifs peuvent avoir une apparence très similaire. Ces cas sont importants, car ils correspondent aux situations qui provoquent des changements d'identité.

Le premier objectif d'apprentissage est une entropie croisée binaire. Pour un logit $s_{i,t}$, on calcule :

$$p_{i,t} = \sigma(s_{i,t}), \quad (29)$$

$$L_{\text{BCE}} = -y \log p_{i,t} - (1 - y) \log(1 - p_{i,t}). \quad (30)$$

Une pondération positive peut être utilisée lorsque les échantillons positifs sont moins nombreux que les négatifs.

Un objectif de classement est aussi ajouté. Pour une même détection, le score de la trajectoire correcte doit être supérieur aux scores des trajectoires incorrectes :

$$s^+ > s^-. \quad (31)$$

La forme utilisée est proche d'un objectif InfoNCE :

$$L_{\text{rank}} = -\log \frac{\exp(s^+/T)}{\sum_j \exp(s_j/T)}. \quad (32)$$

Dans cette expression, T est un paramètre de température.

Une perte auxiliaire est appliquée à la branche de mémoire longue afin qu'elle apprenne aussi à distinguer les paires positives et négatives. Elle est notée L_{long} et prend la forme d'une entropie croisée binaire calculée à partir du logit produit par cette branche. L'objectif global d'entraînement s'écrit alors :

$$L_{\text{train}} = L_{\text{BCE}} + \lambda_{\text{rank}} L_{\text{rank}} + \lambda_{\text{long}} L_{\text{long}}. \quad (33)$$

Dans la configuration retenue, $\lambda_{\text{rank}} = 1$ et $\lambda_{\text{long}} = 1$. La sélection du point de sauvegarde repose sur l'objectif principal de validation :

$$L_{\text{val}} = L_{\text{BCE}} + \lambda_{\text{rank}} L_{\text{rank}}. \quad (34)$$

La perte longue reste donc un terme auxiliaire d'entraînement. Elle n'est pas ajoutée au critère de sélection, car une meilleure classification par la branche longue seule ne garantit pas une meilleure association finale après fusion.

Les détails pratiques de génération des paires, du découpage des séquences et des paramètres d'entraînement sont présentés dans le chapitre d'expérimentations.

3.4 Intégration du score dans l'association existante

Après entraînement, le modèle est chargé pendant l'inférence. Pour chaque paire candidate, il produit un score $s_{i,j}$, transformé en probabilité temporelle :

$$p_{i,j}^{\text{temp}} = \sigma(s_{i,j}). \quad (35)$$

Cette probabilité est convertie en coût temporel :

$$C_{i,j}^{\text{temp}} = 1 - p_{i,j}^{\text{temp}}. \quad (36)$$

Un coût faible signifie que le modèle considère la paire comme plausible.

Ce coût temporel est utilisé avec le coût de référence OSNet. De manière générale, la fusion peut être écrite :

$$C_{i,j}^{\text{fused}} = \text{fusion}(C_{i,j}^{\text{base}}, C_{i,j}^{\text{temp}}). \quad (37)$$

Dans l'implémentation actuelle, la fonction fusion conserve le coût de référence comme terme principal et ajoute une correction temporelle centrée, redimensionnée et limitée :

$$\text{fusion}(C_{i,j}^{base}, C_{i,j}^{temp}) = C_{i,j}^{base} + \Delta C_{i,j}^{temp}, \quad (38)$$

avec

$$\bar{C}_{i,j}^{temp} = C_{i,j}^{temp} - \text{mean}(C^{temp}), \quad (39)$$

$$\Delta C_{i,j}^{temp} = \text{clip}(\beta \bar{C}_{i,j}^{temp}, -c_{\max}, c_{\max}). \quad (40)$$

Le coefficient β adapte l'échelle de la correction à la variabilité des coûts dans l'ensemble candidat, et $c_{\max}$ limite l'influence maximale du modèle appris.

Cette stratégie garde C^{base} comme référence principale tout en permettant au modèle temporel de corriger les associations ambiguës. Après cette fusion, les étapes classiques sont conservées : filtrage géométrique de Kalman puis association hongroise sur C^{fused} . Dans la configuration avec récupération temporelle, le coût temporel intervient aussi pendant le filtrage géométrique : lorsqu'une paire est rejetée par le filtrage géométrique classique, le meilleur candidat temporel peut être conservé si sa distance de Kalman reste dans une marge contrôlée.

3.5 Règle d'acceptation avec récupération temporelle

La Figure 6 détaille le module d'acceptation résumé dans la structure globale. Après la fusion des coûts, la matrice C^{fused} passe par le filtrage géométrique de Kalman avec récupération temporelle, puis par une règle appliquée avant le seuil d'association. Cette règle vérifie d'abord le seuil principal sur le coût fusionné, puis applique une règle de secours temporelle lorsque ce seuil n'est pas satisfait.

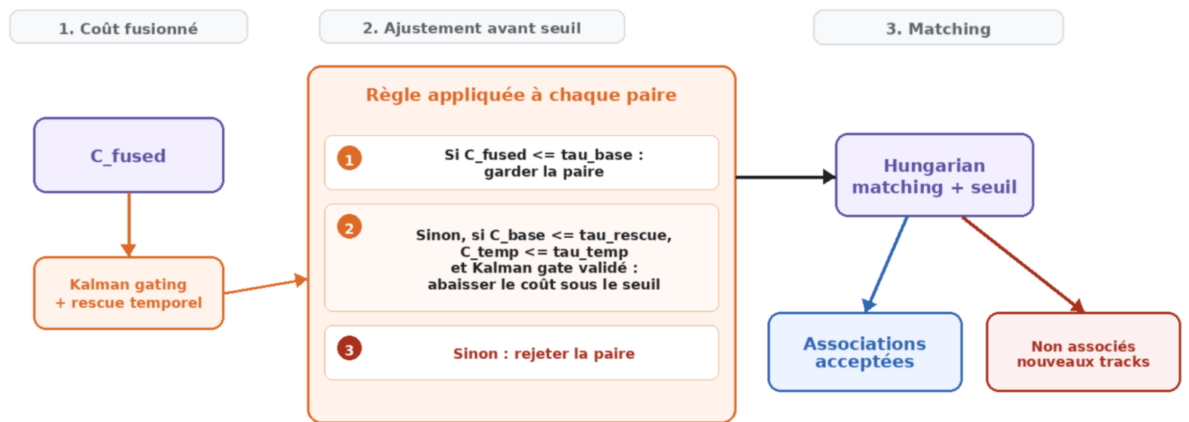


FIGURE 6 – Détail de la règle d'acceptation avec récupération temporelle.

Dans cette règle, chaque association candidate est d’abord évaluée avec le seuil principal :

$$C_{i,j}^{fused} \leq \tau_{base}. \quad (41)$$

Si cette condition est satisfaite, l’association entre la trajectoire i et la détection j reste sous le seuil principal.

Si le seuil principal n’est pas satisfait, la méthode applique une règle de secours temporelle. Cette règle accepte une association seulement si les trois conditions suivantes sont vérifiées :

$$C_{i,j}^{base} \leq \tau_{rescue}, \quad C_{i,j}^{temp} \leq \tau_{temp}, \quad \text{KalmanPorte}(i, j) = 1. \quad (42)$$

Le seuil τ_{base} correspond au seuil principal appliqué au coût fusionné. Le seuil τ_{rescue} fixe le coût de référence maximal encore acceptable pour une association récupérée par récupération temporelle. Le seuil τ_{temp} contrôle la confiance minimale attendue du modèle temporel. Cette seconde étape abaisse le coût des paires qui satisfont ces conditions afin qu’elles puissent être retenues par l’association lorsque le modèle temporel et le filtrage géométrique de Kalman sont cohérents.

3.6 Post-traitement AFLink

Après la règle d’acceptation avec récupération temporelle, FiTAC-MOT peut appliquer AFLink [3] comme post-traitement de trajectoires. AFLink signifie *modèle de liaison sans apparence*. Son objectif est de connecter deux *fragments de trajectoire* séparés qui peuvent correspondre à la même personne.

Pendant l’entraînement, AFLink écarte les informations d’apparence, notamment les vecteurs d’apparence ReID. Le modèle utilise les informations spatio-temporelles des fragments de trajectoire, comme les indices de trames et la position du centre des boîtes. Il apprend ainsi à prédire si deux fragments de trajectoire appartiennent à la même identité réelle. Le module produit alors un score de liaison entre les deux fragments de trajectoire :

$$p_{a,b}^{link} = F_{\text{AFLink}}(T_a, T_b). \quad (43)$$

où T_a et T_b désignent deux fragments de trajectoire candidats, F_{AFLink} le modèle de liaison appris, et $p_{a,b}^{link}$ la probabilité que ces deux fragments appartiennent à la même identité. Lorsque ce score dépasse un seuil d’acceptation, les deux fragments sont fusionnés sous le même identifiant.

Dans notre pipeline, AFLink complète le score temporel appris. Le TemporalAttentionScorer agit pendant l’association détection-trajectoire, puis AFLink intervient après

le suivi pour réduire la fragmentation des trajectoires restantes. Cette étape conserve les détections et le coût C^{fused} produits pendant l'association. Dans nos résultats, elle réduit le nombre d'identités prédites et diminue les changements d'identité.

4 Intégration dans le pipeline fisheye/gnomonique

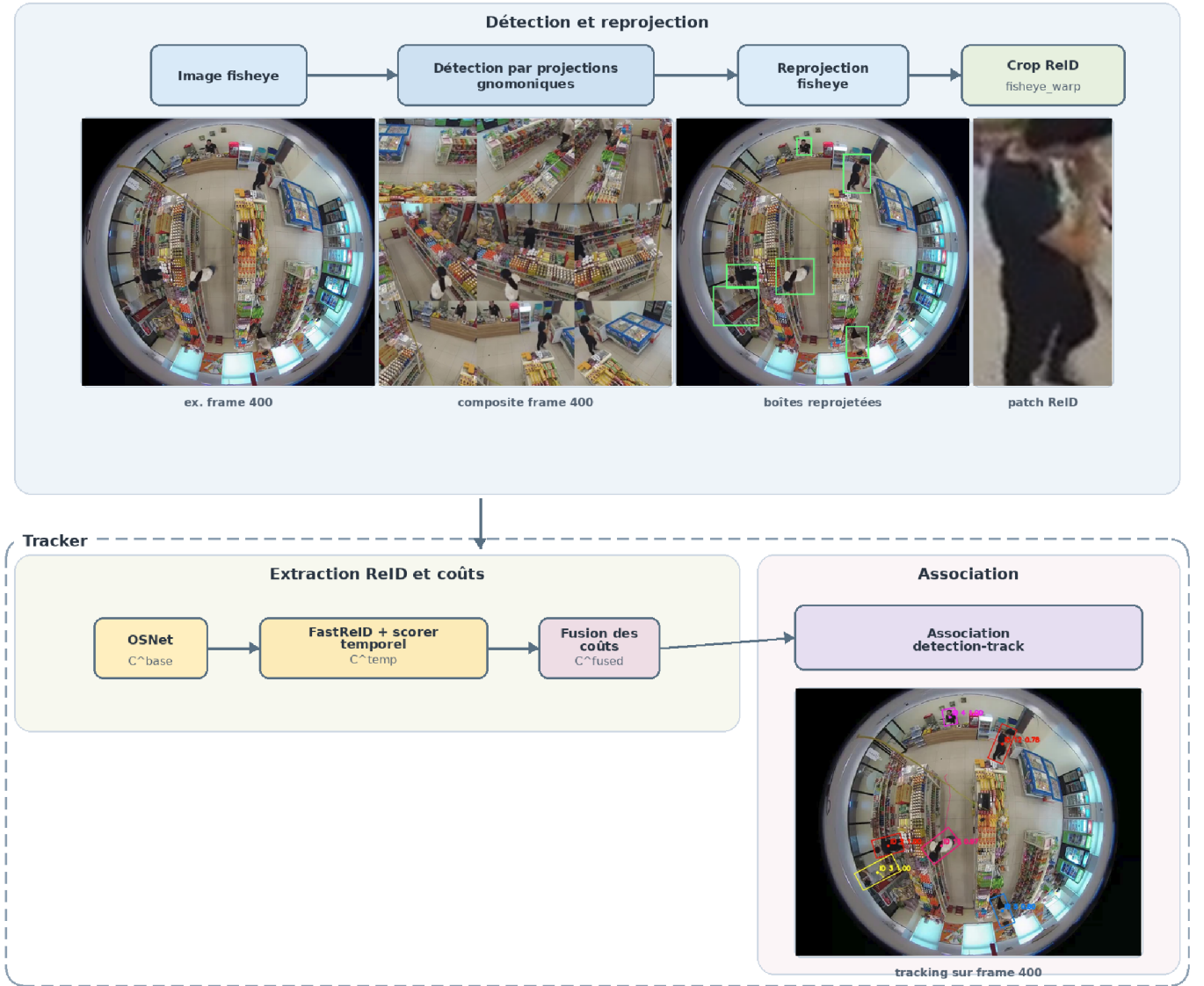


FIGURE 7 – Flux global du pipeline fisheye/gnomonique et du module de suivi.

4.1 Position du traqueur dans le pipeline global

Le traqueur intervient après la détection fisheye. Dans le pipeline global, l'image fisheye est d'abord traitée par projections gnomoniques afin de réduire l'effet des distorsions sur la détection. Le détecteur produit ensuite des boîtes de piétons sur les projections ou sur l'image composite. Ces détections sont finalement reprojctées dans le repère de l'image fisheye.

Le module de suivi exploite les sorties du détecteur pour construire les trajectoires. À chaque trame, il reçoit les détections courantes, construit la région ReID selon la configuration retenue, extrait les vecteurs d'apparence OSNet et FastReID, puis associe les détections aux trajectoires existantes à partir de C^{fused} et de la règle d'acceptation. Le système ne produit donc plus seulement des boîtes indépendantes trame par trame, mais des trajectoires avec des identifiants.

La Figure 7 résume ce flux global, depuis l'image fisheye jusqu'à l'association entre détections et trajectoires.

4.2 Adaptation des détections et extraction ReID dans le pipeline fisheye

Dans le pipeline global, les détections reprojctées dans le repère fisheye sont d'abord converties en objets compatibles avec le traqueur, notamment sous la forme de boîtes *tlwh* utilisées par le filtre de Kalman et le *filtrage géométrique*.

L'extraction des vecteurs d'apparence repose ensuite sur la construction d'une région ReID à partir de la détection courante. Le principe de la représentation `fisheye_warp` et son intérêt par rapport à un recadrage aligné sur les axes ont été présentés en Section 2.2. La région extraite est utilisée comme entrée commune à OSNet pour la branche de référence et à FastReID pour la branche temporelle.

Ainsi, dans l'intégration complète du système, ce module joue le rôle d'interface entre les détections fisheye reprojctées, la représentation géométrique du traqueur et les branches d'extraction ReID.

5 Conclusion

Ce chapitre a présenté les principales contributions réalisées pendant le stage. Le point de départ est une configuration de référence de suivi de type DeepSORT, associée à des

vecteurs d'apparence extraits avec OSNet. Cette configuration de référence permet d'intégrer le module de suivi dans le pipeline de détection fisheye/gnomonique existant, tout en conservant une structure classique de suivi par détection, fondée sur la prédiction de mouvement, le coût d'apparence OSNet et l'association hongroise.

Nous avons ensuite proposé un modèle appris d'association temporelle, appelé TemporalAttentionScorer. Ce modèle prend comme entrée une paire candidate détection-trajectoire. Il combine la caractéristique FastReID de la détection courante, la mémoire courte FastReID de la trajectoire et sa mémoire longue FastReID, puis produit un coût temporel C^{temp} . Ce coût intervient dans la fusion avec C^{base} , dans la récupération temporelle au niveau du filtrage géométrique et dans la règle d'acceptation avant le seuil d'association, tout en conservant le filtrage géométrique de Kalman et l'association hongroise. FiTAC-MOT applique aussi AFLink comme post-traitement afin de relier certains fragments de trajectoire après le suivi.

Enfin, ce chapitre a décrit l'intégration de ces modules dans le pipeline fisheye/gnomonique. Dans l'ensemble, les contributions améliorent la stabilité de l'association d'identité lorsque l'apparence des piétons varie au cours du temps. Le chapitre suivant présente les expériences menées pour analyser l'effet de ces différents modules sur les performances de suivi.

Expérimentations et résultats

1 Introduction

Ce chapitre présente la conception des expériences et l’analyse des résultats de FiTAC-MOT. L’objectif est d’évaluer si le score temporel appris et le post-traitement AFLink aident le traqueur lorsque l’apparence des piétons change au cours d’une séquence. Dans ce contexte, l’amélioration attendue concerne surtout la réduction des changements d’identité et le maintien d’une identité cohérente pour chaque personne.

Les résultats sont donc analysés à partir de métriques de suivi multi-objets liées à l’identité, comme IDF1, IDSW, HOTA et AssA.

2 Données et métriques

Cette section présente les jeux de données utilisés, le format des annotations, ainsi que la distribution entre entraînement, validation et test.

2.1 Jeux de données

Les données d’entraînement comprennent les scènes `jewelry_store` et `jewelry_store_2` du jeu de données WEPDToF, ainsi que les séquences `capfica3` et `capfica5` de notre jeu de données CapFiCA (Caplogy Fisheye Change Appearance). La validation utilise `capfica1`. L’ensemble de test correspond à `capfica4`, qui n’est pas utilisé pendant l’entraînement ni pendant la sélection du modèle.

2.1.1 WEPDToF

WEPDToF [10] est un jeu de données orienté vers la détection et le suivi de piétons en contexte fisheye. Il contient plusieurs scènes intérieures, par exemple `jewelry_store`,

`convenience_store`, des bureaux et des entrepôts. L'échelle, l'orientation et la forme apparente des piétons peuvent varier selon leur position dans l'image.



(a) Exemple de la scène `jewelry_store`.



(b) Exemple de la scène `jewelry_store_2`.

FIGURE 1 – Exemples de scènes WEPDToF utilisées pour l'entraînement.

Dans ce travail, nous utilisons surtout les scènes `jewelry_store` et `jewelry_store_2` pour l'entraînement. Ces scènes contiennent des déplacements de piétons dans un environnement intérieur partiellement ouvert. Elles permettent de générer des paires positives et négatives pour apprendre l'association temporelle. Les images et les annotations fournies par WEPDToF sont utilisées pour construire les paires détection-trajectoire nécessaires à l'entraînement du *TemporalAttentionScorer*.

2.1.2 CapFiCA

CapFiCA, pour *Caplogy Fisheye Change Appearance*, correspond aux séquences fisheye que nous avons filmées et préparées dans le bureau de Caplogy. Ce jeu de données est plus proche du contexte applicatif du projet. Il met l'accent sur les changements d'apparence observés lorsque les piétons se déplacent dans l'image fisheye : changement de vêtements, changement d'accessoires comme les lunettes de soleil ou le chapeau, modification de l'apparence propre de la personne avec les cheveux attachés ou détachés, et changement de posture avec des actions comme s'accroupir, se relever ou s'asseoir en interaction avec la scène.

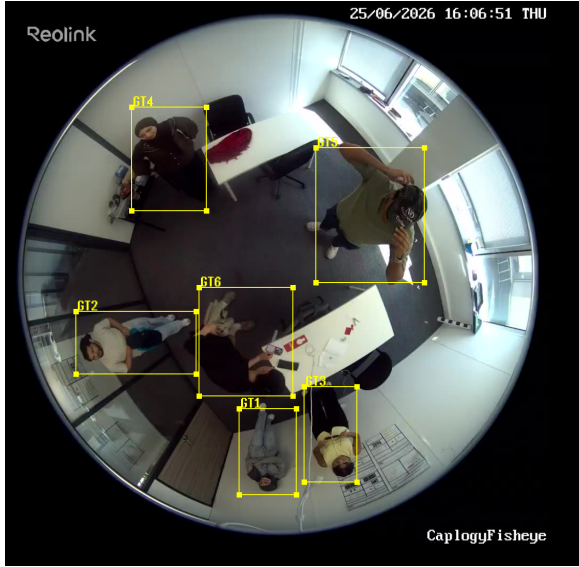
Le Tableau 1 résume les informations principales des séquences CapFiCA utilisées dans ce travail.

Jeu de données	Trames	Personnes
capfica1	552	6
capfica2	784	6
capfica3	760	8
capfica4	1076	5
capfica5	1176	5
COMBINÉ	4348	5–8

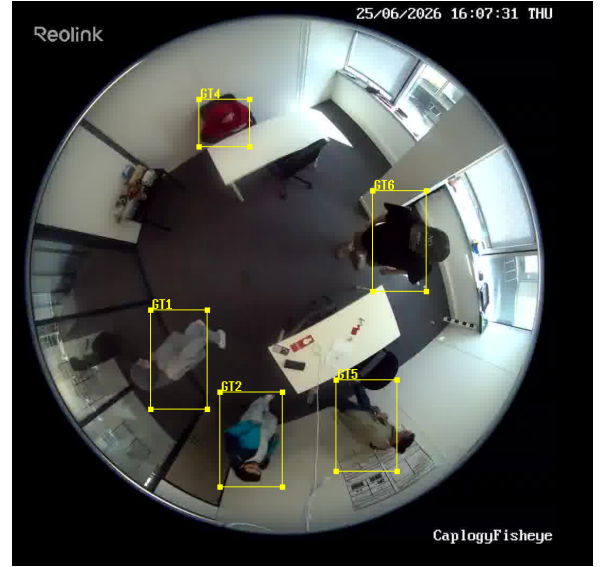
TABLE 1 – Informations principales des séquences CapFiCA utilisées.

Les comportements présents dans ces séquences comprennent : changement de veste, port de sac à dos, port d'écharpe, port de lunettes de soleil, port de chapeau, assis/debout, accroupissement et relèvement, rotation, utilisation d'un parapluie et croisement entre personnes.

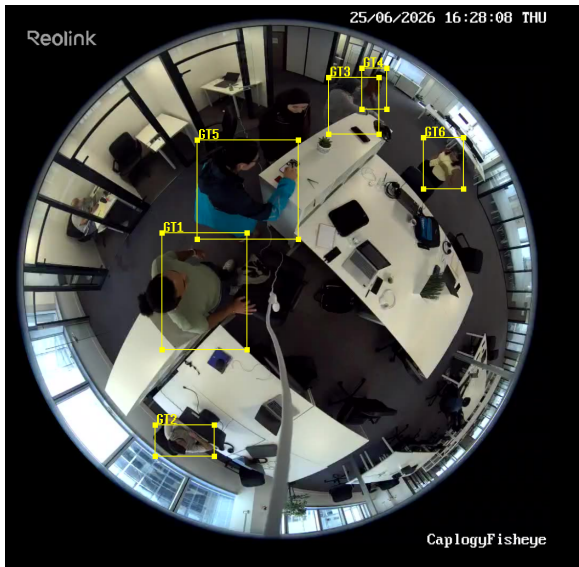
Pour rendre ces séquences utilisables en évaluation de suivi, nous avons produit des annotations nous-mêmes. Les vidéos sélectionnées couvrent un espace ouvert et un espace fermé, afin d'observer le comportement du traqueur dans des conditions de déplacement différentes.



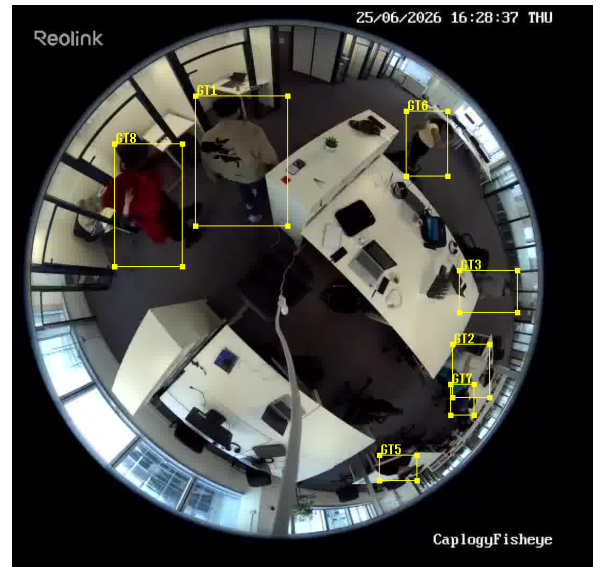
(a) Séquence CapFiCA 1, trame a.



(b) Séquence CapFiCA 1, trame b.



(c) Séquence CapFiCA 2, trame c.



(d) Séquence CapFiCA 2, trame d.

FIGURE 2 – Exemples de séquences CapFiCA. Ces images montrent que l'apparence d'un même piéton peut changer entre différentes trames.

2.2 Métriques d'évaluation

L'évaluation est réalisée avec TrackEval [11], un outil qui fournit plusieurs métriques classiques pour le suivi multi-objets, notamment HOTA [12]. Dans ce travail, nous retenons surtout IDF1, IDSW, AssA, DetA, MOTA et HOTA. Ces métriques permettent d'analyser la stabilité des identités et la qualité de l'association, ce qui correspond directement au

problème étudié.

2.2.1 IDF1 (Identity F1 Score)

IDF1 mesure la cohérence entre les identités prédites et les identités réelles sur l'ensemble de la séquence [13]. La formule est :

$$IDF1 = \frac{2 \times IDTP}{2 \times IDTP + IDFP + IDFN}. \quad (1)$$

Dans cette expression, $IDTP$ désigne les détections dont l'identité est correctement associée, $IDFP$ les détections attribuées à une mauvaise identité, et $IDFN$ les détections réelles dont l'identité n'est pas correctement retrouvée.

Une valeur élevée indique que le traqueur conserve mieux les identités au cours du temps. Pour notre sujet, IDF1 est une métrique centrale, car elle mesure directement la continuité d'identité.

2.2.2 IDSW (Identity Switches)

IDSW compte le nombre de changements d'identité. Un changement d'identité apparaît lorsqu'un même piéton réel reçoit plusieurs identifiants prédits au cours de la séquence. Plus cette valeur est faible, moins le traqueur produit de ruptures d'identité.

2.2.3 AssA (Association Accuracy)

AssA évalue la qualité de l'association des trajectoires lorsque les détections sont déjà correctement appariées aux objets réels. Elle s'écrit :

$$AssA = \frac{AssTP}{AssTP + AssFP + AssFN}. \quad (2)$$

Ici, $AssTP$ correspond aux associations correctes à l'intérieur d'une même trajectoire réelle, $AssFP$ aux associations ajoutées à tort, et $AssFN$ aux associations manquées.

Cette métrique se concentre sur la cohérence interne des trajectoires. Elle est particulièrement utile pour analyser notre modèle temporel, car celui-ci agit au niveau de l'association entre détections et trajectoires.

2.2.4 DetA (Detection Accuracy)

DetA mesure la qualité des détections appariées aux objets réels :

$$DetA = \frac{DetTP}{DetTP + DetFP + DetFN}. \quad (3)$$

Dans cette formule, $DetTP$ désigne les détections correctement associées à un objet réel, $DetFP$ les fausses détections, et $DetFN$ les objets réels non déteints.

Cette métrique permet de distinguer les limites liées à l'entrée du traqueur. Si $DetA$ est faible, les performances globales peuvent rester limitées même lorsque l'association temporelle s'améliore.

2.2.5 MOTA (Multiple Object Tracking Accuracy)

MOTA est une métrique globale couramment utilisée en suivi multi-objets [14] :

$$MOTA = 1 - \frac{FN + FP + IDSW}{GT}. \quad (4)$$

Dans cette formule, FN représente les objets manqués, FP les fausses détections, $IDSW$ les changements d'identité, et GT le nombre total d'objets annotés.

MOTA reste utile pour mesurer l'erreur globale, mais elle est fortement influencée par les erreurs de détection.

2.2.6 HOTA (Higher Order Tracking Accuracy)

De manière simplifiée, HOTA combine la qualité de détection et la qualité d'association [12] :

$$HOTA = \sqrt{DetA \times AssA}. \quad (5)$$

Une valeur élevée indique un bon équilibre entre la détection des piétons et la continuité des trajectoires. Cette métrique donne une vision plus équilibrée du traqueur que MOTA.

* Le FPS est reporté à titre indicatif, mais il n'entre pas dans l'objectif principal de ce stage.

3 Préparation des données et configurations

3.1 Annotation de CapFiCA

L'annotation de CapFiCA repose sur une correction manuelle des résultats produits par un traqueur. Le processus suivi est le suivant :

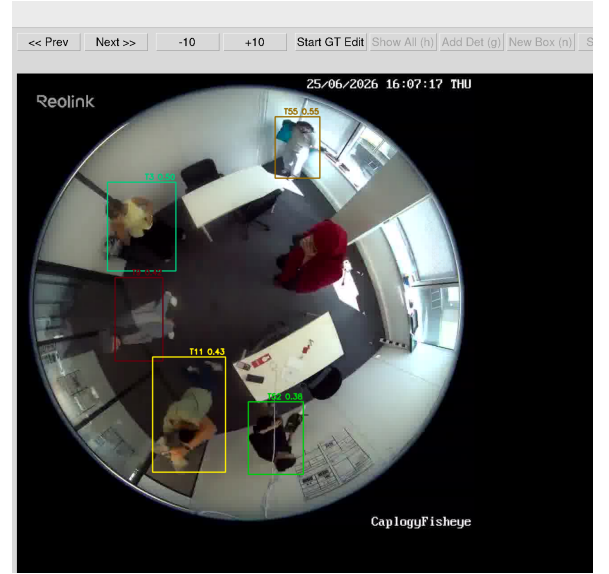
1. **Génération d'un résultat initial de suivi.** Nous lançons d'abord le traqueur existant sur les vidéos CapFiCA. Le résultat obtenu sert d'annotation initiale. Il contient

déjà une grande partie des boîtes et des identifiants de trajectoire. Certaines erreurs restent présentes, notamment des détections manquées, des trajectoires fragmentées et des changements d'identité. Ces erreurs viennent principalement de la déformation fisheye, des variations d'apparence et des limites du détecteur utilisé.

2. **Chargement dans l'éditeur GT.** Les résultats du traqueur sont ensuite ouverts dans notre éditeur GT. Cet outil permet de parcourir la séquence image par image, de visualiser les trajectoires, de modifier les identifiants, d'ajouter des boîtes manquantes et d'exporter un fichier de vérité terrain.



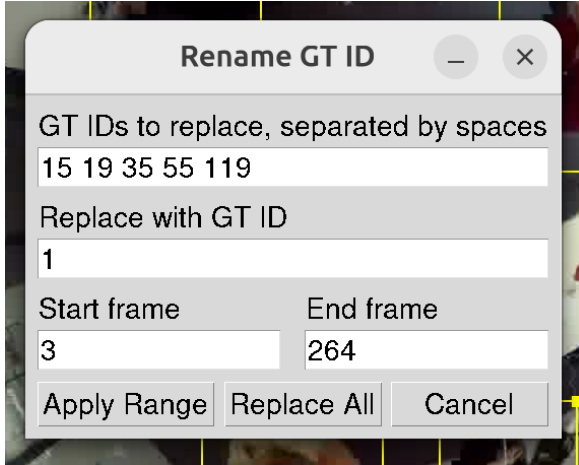
(a) Résultat initial du traqueur.



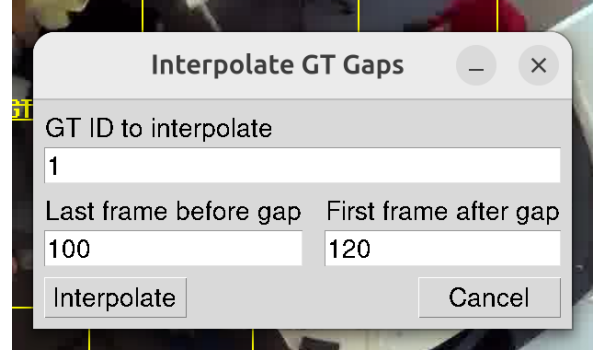
(b) Chargement dans l'éditeur GT.

FIGURE 3 – Premières étapes de l'annotation : génération d'une annotation initiale, puis ouverture des résultats dans l'éditeur GT pour commencer la correction.

3. **Correction des identifiants.** Nous vérifions d'abord la cohérence de chaque identité. Lorsqu'un même piéton est séparé en plusieurs fragments de trajectoire, les identifiants sont modifiés en groupe afin de les rattacher au même ID. Cette étape corrige principalement les changements d'identité produits par le traqueur initial.
4. **Complétion des détections manquées.** Dans certaines trames, le traqueur ne détecte pas le piéton et la trajectoire présente une interruption. Pour ces cas, nous utilisons la fonction d'interpolation de l'éditeur GT. Les boîtes manquantes sont générées à partir des positions annotées avant et après l'interruption. Les résultats interpolés sont ensuite vérifiés manuellement pour s'assurer que les boîtes restent alignées avec la position réelle du piéton.



(a) Correction des identifiants.



(b) Interpolation des boîtes manquantes.

FIGURE 4 – Étapes de correction manuelle : regroupement des fragments appartenant au même piéton, puis complétion des détections manquées par interpolation.

5. **Export de la vérité terrain.** Après correction des identités et complétion des boîtes manquantes, les annotations sont exportées au format MOT. Chaque ligne contient l'indice de trame, l'identifiant de la personne, les coordonnées de la boîte englobante et les informations nécessaires à l'évaluation. Ce format est utilisé directement avec TrackEval.

Une version complémentaire des annotations de `capfica4` a ensuite été corrigée sous forme de boîtes orientées. Elle conserve, pour chaque piéton, le centre, la largeur, la hauteur, l'angle et les quatre coins de la boîte. Ces annotations permettent une évaluation mieux adaptée à la géométrie radiale des personnes dans l'image fisheye. Elles sont utilisées dans une évaluation complémentaire fondée sur l'IoU polygonale. Les résultats obtenus avec ce protocole orienté sont présentés séparément des résultats TrackEval historiques fondés sur des boîtes horizontales.

3.2 Configurations évaluées

Pour rendre les comparaisons cohérentes, toutes les configurations sont exécutées avec le même pipeline de suivi. Les différences entre expériences concernent principalement l'utilisation du modèle temporel, du mécanisme de récupération temporelle et du post-traitement AFLink. Les paramètres du traqueur présentés ci-dessous correspondent à la configuration finale utilisée dans ce travail.

3.2.1 Détection et projection fisheye

Les séquences CapFiCA utilisées en test sont d’abord traitées par le pipeline de projection gnomonique. La configuration de projection et le détecteur sont les suivants :

- configuration composite : `wide#3-y0.25-640` ;
- détecteur : YOLOv9e [15] ;
- seuil de confiance des détections : 0.25 ;
- matériel utilisé : `cuda` : NVIDIA RTX4060.

3.2.2 Configuration de référence de suivi

La configuration de référence suit une organisation DeepSORT/StrongSORT. Le filtre de Kalman fournit la prédiction géométrique, tandis que le coût d’apparence est calculé avec OSNet. Les paramètres utilisés sont :

- backend ReID de la configuration de référence : OSNet ;
- modèle OSNet : `osnet_x1_0` ;
- poids OSNet : `osnet_x1_0_imagenet.pth` ;
- seuil d’association : 0.2 ;
- distance IoU maximale : 0.7 ;
- `max_age` : 100 ;
- nombre minimal de détections avant confirmation (`n_init`) : 3 ;
- taille maximale de la galerie d’apparence (`nn_budget`) : 100.

3.2.3 Encodeur temporel FastReID

Le modèle temporel utilise FastReID pour extraire les vecteurs d’apparence utilisés par le *TemporalAttentionScorer*. FastReID sert uniquement à cette branche temporelle. Les paramètres utilisés sont :

- backend ReID temporel : FastReID ;
- configuration FastReID : `bagtricks_S50.yml` ;
- poids FastReID : `duke_bot_S50.pth` ;
- taille de lot ReID temporel : 32 ;
- matériel utilisé : `cuda` : NVIDIA RTX4060.

Toutes les régions ReID sont extraites avec le mode `fisheye_warp`. Les vecteurs d’apparence OSNet et FastReID correspondent aux équations 1 et 2.

3.2.4 Modèle temporel appris

Le modèle temporel chargé pendant les expériences correspond au point de sauvegarde suivant :

- point de sauvegarde : `retrain_val_bce_rank/best.pt` ;
- époque retenue : 23 ;
- critère de sélection : $L_{\text{BCE}} + L_{\text{rank}}$.

Ce modèle est entraîné avec `jewelry_store`, `jewelry_store_2`, `capfica3` et `capfica5`. La sélection du modèle est réalisée avec `capfica1`. Les tests rapportés dans cette section sont réalisés sur `capfica4`, qui n'est pas utilisé pendant l'entraînement ni pendant la validation.

Les paramètres de fusion utilisés au moment de l'inférence sont :

- `learned_temporal_alpha` : 0.35 ;
- `learned_temporal_min_scale` : 0.02 ;
- `learned_temporal_max_correction` : 0.02 ;

Ces paramètres contrôlent l'amplitude de la correction temporelle ajoutée au coût d'apparence de référence, définie dans les équations 39 et 40.

3.2.5 Récupération temporelle

La configuration finale active le mécanisme de récupération temporelle. Il permet de récupérer certaines associations rejetées par le seuil principal lorsque le modèle temporel donne un coût très faible. Les seuils utilisés sont :

- seuil d'acceptation temporelle : $\tau_{temp} = 0.2$;
- seuil de récupération temporelle sur le coût de référence : $\tau_{rescue} = 0.35$.

Ces deux seuils correspondent à la règle de récupération temporelle donnée dans l'équation 42. La configuration finale applique ensuite AFLink sur les trajectoires produites par le traqueur.

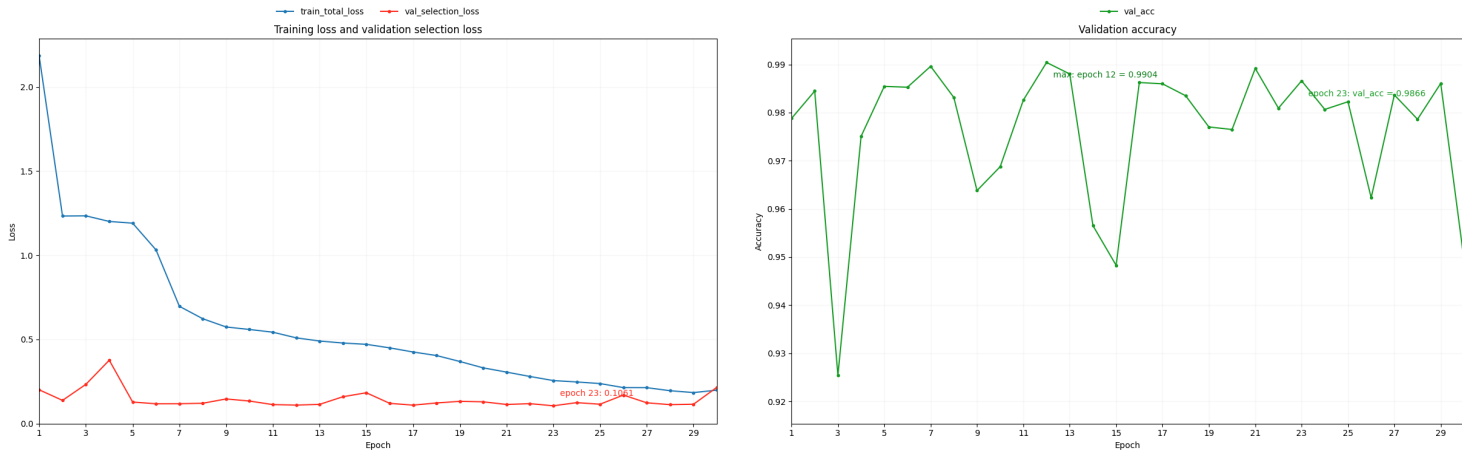
Les paramètres AFLink conservés sont un intervalle temporel de 0 à 80 trames, une distance spatiale maximale de 250 pixels et un seuil de liaison de 0.2. Toutes les valeurs de cette section correspondent aux valeurs par défaut du profil final utilisé pour les résultats rapportés.

3.2.6 Reproductibilité de la configuration

Les paramètres finaux sont centralisés dans le profil `fitacmot_capfica_final.yaml`. Le dépôt fournit aussi un script d'installation, une commande de vérification des dépendances et des poids, ainsi qu'une interface commune pour l'inférence et l'évaluation. Les sorties principales sont les trajectoires au format MOT, les trajectoires orientées et un fichier qui enregistre la configuration effectivement utilisée. Un test de reproductibilité sur dix trames, réalisé dans un environnement CPU nouvellement créé, a permis de vérifier le chargement des cinq modèles et l'exécution complète du pipeline.

3.3 L'entraînement du modèle temporel appris

Avant d'analyser les résultats de suivi, nous vérifions aussi le comportement de l'entraînement du modèle temporel appris. La Figure 5 présente l'évolution de la perte totale et de la précision de validation.



(a) Perte totale d'entraînement et perte de sélection en validation.

(b) Précision de validation.

FIGURE 5 – Courbes d'entraînement du modèle temporel appris. La courbe bleue correspond à $L_{\text{train}} = L_{\text{BCE}} + L_{\text{rank}} + L_{\text{long}}$, tandis que la courbe rouge correspond au critère de validation $L_{\text{val}} = L_{\text{BCE}} + L_{\text{rank}}$. Le meilleur point de sauvegarde est sélectionné à l'époque 23, avec une perte de sélection de 0.1061.

Le choix du meilleur checkpoint : l'époque 12 obtient la précision de validation la plus élevée, tandis que l'époque 23 minimise le critère principal de validation. Par exemple, deux paires positives avec des scores de 0,51 et 0,99 sont toutes les deux comptées comme

correctes par la précision. En revanche, l'entropie croisée favorise davantage le score 0,99, car il exprime une confiance plus forte dans la bonne classe. Lorsque la précision est déjà proche de la saturation, la perte de validation distingue donc mieux la qualité probabiliste et le classement des candidats.

Au début de l'entraînement, la perte sur l'ensemble de validation est déjà faible, puis elle fluctue par la suite : comme la plupart des paires de l'ensemble de validation sont faciles à distinguer, le modèle apprend rapidement à séparer les exemples positifs des exemples négatifs. L'évolution de la perte sur l'ensemble de validation dépend principalement d'un petit nombre de paires difficiles à distinguer. Si, au cours d'une époque donnée, le modèle attribue à ces paires des scores trop confiants et incorrects, la perte de validation peut augmenter, par exemple vers la 20e époque. Lorsque les paramètres changent légèrement à la prochaine époque, ces scores peuvent redevenir plus cohérents, et la perte de validation diminue.

La précision de validation ne présente pas de tendance croissante, car elle est déjà proche de la saturation. Pour la plupart des époques d'entraînement, la précision de validation se situe entre 0,97 et 0,99, avec une valeur maximale de 0,9904. Par conséquent, le modèle est capable de classer correctement la plupart des paires de données assez tôt. Les fluctuations restantes proviennent principalement des paires difficiles à distinguer, qui peuvent être classées correctement lors d'une époque, mais de manière erronée lors de l'époque suivante (33). De plus, les paires de validation ne sont pas totalement indépendantes : beaucoup d'entre elles viennent de la même image ou du même segment de trajectoire. Si le modèle traite mal un segment difficile, plusieurs paires sont alors affectées simultanément. C'est une instabilité sur un ensemble de validation de petite taille, et pas un échec global de l'entraînement.

3.4 Étude de la mémoire à long terme

La Figure 6 sépare les trois composantes de l'objectif d'entraînement. La diminution de L_{long} montre que la branche longue apprend effectivement à distinguer les paires positives et négatives. Cette diminution est cependant plus lente que celle des termes BCE et InfoNCE.

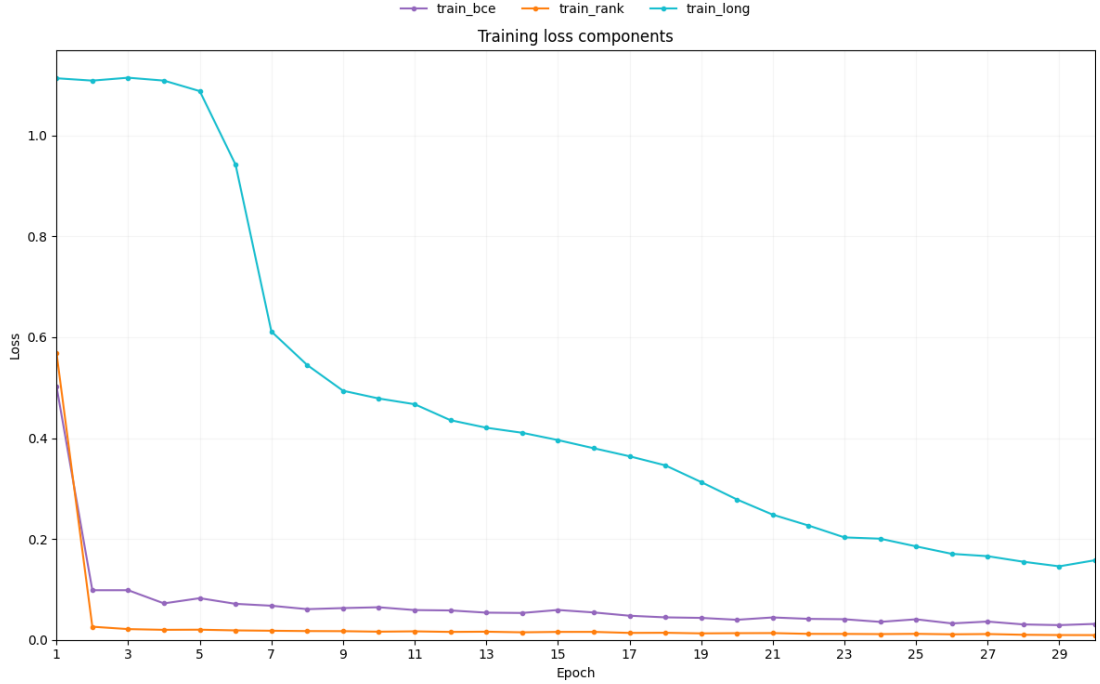


FIGURE 6 – Composantes de la perte d’entraînement : BCE, classement InfoNCE et perte auxiliaire de la mémoire longue.

Plusieurs variantes ont été testées : ajout de la perte auxiliaire longue, fusion explicite du score long terme, modification du critère de sélection et reconstruction des paires avec des annotations orientées. Les deux réentraînements réalisés avec les nouvelles paires n’ont pas amélioré les mesures de suivi sur `capfica4`. Le premier obtient un rHOTA de 28.067, une rAssA de 13.137, un IDF1 de 29.486 et 58 changements d’identité. La variante utilisant uniquement les annotations orientées réelles pour les nouvelles séquences obtient un rHOTA de 30.049, une rAssA de 15.061, un IDF1 de 29.962 et 57 changements d’identité. Ces valeurs restent inférieures à celles du point de sauvegarde final retenu.

Cette étude montre qu’une baisse de la perte auxiliaire longue ne garantit pas une meilleure association finale. Une utilisation plus efficace de la LTM nécessiterait de l’activer davantage lorsque la mémoire courte est incertaine, ou d’apprendre directement la fusion entre les deux branches.

4 Expérimentations

Cette section présente les expériences utilisées pour évaluer FiTAC-MOT dans des séquences fisheye avec changements d’apparence. Les résultats ci-dessous utilisent le nou-

veau découpage des données, résumé dans le Tableau 2. L’objectif est d’évaluer l’effet du modèle temporel, de la récupération temporelle et d’AFLink sur une séquence de test indépendante.

Deux protocoles de localisation sont distingués. Les ablations et les comparaisons historiques utilisent le protocole MOT standard avec des boîtes horizontales et une IoU horizontale. Une évaluation complémentaire utilise les nouvelles annotations orientées de `capfica4` et une IoU polygonale. Les valeurs de HOTA, DetA et AssA dépendent des appariements produits par le seuil d’IoU ; elles ne doivent donc pas être comparées directement entre ces deux protocoles.

Ensemble	Séquences utilisées
Entraînement	<code>jewelry_store</code> , <code>jewelry_store_2</code> , <code>capfica3</code> , <code>capfica5</code>
Validation	<code>capfica1</code>
Test	<code>capfica4</code>

TABLE 2 – Nouveau découpage utilisé pour l’entraînement, la validation et le test.

Nous commençons par une étude d’ablation, qui compare la configuration de référence OSNet avec les différentes variantes temporelles du traqueur. Nous étudions ensuite l’influence du choix des encodeurs ReID. Nous comparons ensuite FiTAC-MOT avec ByteTrack et BoT-SORT sur la même séquence de test. Enfin, une dernière expérience vérifie la généralisation sur plusieurs scènes WEPDToF.

4.1 Expérience 1 : étude d’ablation du modèle temporel

4.1.1 Protocole expérimental

Cette expérience analyse progressivement l’effet des composants ajoutés au traqueur. Les tests sont réalisés sur la séquence `capfica4`. Nous partons de la configuration de référence OSNet, qui utilise uniquement le coût d’apparence OSNet pour l’association. Nous ajoutons ensuite AFLink à la configuration de référence, puis le modèle temporel FastReID avec fusion temporelle, le mécanisme de récupération temporelle et enfin AFLink dans la configuration complète.

Cette ablation a été réalisée avant la dernière sélection du point de sauvegarde. Elle sert à mesurer la contribution relative des modules. La configuration finale réévaluée avec le point de sauvegarde de l’époque 23 est rapportée séparément dans le Tableau 4.

4.1.2 Résultats

Configuration	HOTA	DetA	AssA	MOTA	IDF1	IDSW	Pred. IDs
Configuration de référence OSNet	34.301	82.864	14.205	83.579	26.738	90	81
Configuration de référence OSNet + AFLink	38.784	82.582	18.217	83.844	32.504	76	62
+ Fusion temporelle sans récupération temporelle	34.381	85.735	13.789	85.242	24.869	71	63
+ Fusion temporelle + récupération temporelle	38.398	86.611	17.023	86.451	27.423	59	53
FiTAC-MOT	55.470	86.582	35.538	86.697	47.969	46	36

TABLE 3 – Étude d’ablation du modèle temporel sur **capfica4**, avec le point de sauvegarde utilisé pendant l’ablation et le protocole horizontal historique.

4.1.3 Discussion

- Le modèle temporel améliore la configuration de référence OSNet sur **capfica4**. Avec la récupération temporelle, les changements d’identité passent de 90 à 59 et le nombre d’identités prédites passe de 81 à 53.
- **AFLink** : Dans ces scènes, une même personne peut être divisée en plusieurs fragments de trajectoire courts à cause des changements d’apparence ou des variations de point de vue liées à la caméra fisheye. AFLink peut alors reconnecter certains de ces fragments à partir de leurs relations spatio-temporelles. Cela explique l’amélioration nette de l’IDF1, de l’AssA et du nombre d’identités prédites.
- Lorsque l’apparence change, le coût de référence calculé avec OSNet peut augmenter, même pour une paire détection-trajectoire correcte. Cette paire peut alors être rejetée par le seuil d’association classique. Le mécanisme de récupération temporelle permet de récupérer une partie de ces associations candidates lorsque le modèle temporel les considère encore cohérentes.
- Dans cette ablation, la meilleure configuration combine la fusion temporelle, la récupération temporelle et AFLink. HOTA atteint 55.470, AssA 35.538 et IDF1 47.969, avec 46 changements d’identité et 36 identités prédites.

4.1.4 Mise à jour avec le point de sauvegarde final

Après la sélection du point de sauvegarde de l’époque 23, la configuration complète a été réévaluée avec le protocole horizontal historique. Le Tableau 4 distingue le traqueur en ligne de son post-traitement AFLink.

Configuration	HOTA	DetA	AssA	MOTA	IDF1	IDSW	IDs prédits
FiTAC-MOT sans AFLink	48.673	86.976	27.238	86.640	40.245	55	49
FiTAC-MOT avec AFLink	61.423	86.976	43.377	86.848	55.837	44	36

TABLE 4 – Résultats du point de sauvegarde final sur `capfica4` avec le protocole horizontal historique.

AFLink ne modifie pas les détections : DetA reste donc identique. Il reconnecte certains fragments de trajectoire, ce qui augmente AssA et IDF1, réduit les changements d’identité de 55 à 44 et ramène le nombre d’identités prédites de 49 à 36. Ce résultat confirme le caractère complémentaire de l’association en ligne et du post-traitement hors ligne.

4.2 Expérience 2 : choix des encodeurs d’apparence

4.2.1 Protocole expérimental

Cette expérience étudie l’influence du choix des encodeurs ReID. Deux choix sont testés pour le coût d’apparence de référence : OSNet et FastReID. Deux choix sont aussi testés pour le modèle temporel : OSNet et FastReID. Les combinaisons évaluées sont résumées dans le Tableau 5.

Combinaison	Encodeur de référence	Encodeur temporel
C1	OSNet	OSNet
C2	OSNet	FastReID
C3	FastReID	OSNet
C4	FastReID	FastReID

TABLE 5 – Combinaisons d’encodeurs évaluées.

Toutes les configurations sont testées sur `capfica4`. Nous analysons d’abord les résultats avant AFLink, puis les résultats après AFLink, car FiTAC-MOT inclut ce post-traitement dans sa configuration finale.

4.2.2 Résultats

Combinaison	HOTA	DetA	AssA	MOTA	IDF1	IDSW	Pred. IDs	FP
C1	38.602	84.816	17.570	85.582	30.249	72	56	19
C2	38.398	86.611	17.023	86.451	27.423	59	53	11
C3	41.080	83.611	20.197	85.903	34.418	67	22	74
C4	38.649	83.987	17.799	86.130	30.881	65	21	65

TABLE 6 – Comparaison des combinaisons d’encodeurs avant AFLink sur `capfica4`, avec le protocole horizontal historique.

Configuration	HOTA	DetA	AssA	MOTA	IDF1	IDSW	Pred. IDs	FP
C1 + AFLink	45.804	84.816	24.740	85.752	39.372	63	44	19
C2 + AFLink	55.470	86.582	35.538	86.697	47.969	46	36	11
C3 + AFLink	41.145	83.611	20.261	85.922	34.418	66	21	74
C4 + AFLink	39.530	83.986	18.620	86.168	32.633	63	19	65

TABLE 7 – Comparaison des combinaisons d’encodeurs après AFLink sur `capfica4`, avec le protocole horizontal historique.

4.2.3 Discussion

Avant AFLink, C3 obtient le meilleur HOTA, la meilleure AssA et le meilleur IDF1. C2 obtient cependant le meilleur DetA, la meilleure MOTA et le plus faible nombre de changements d’identité. Les configurations utilisant FastReID comme encodeur de référence produisent beaucoup moins d’identités prédites, mais elles augmentent aussi le nombre de faux positifs dans ce test.

Après AFLink, C2 devient la configuration la plus intéressante pour notre objectif. Cela peut s’expliquer par le fait qu’avant AFLink, C2 laisse davantage de fragments de trajectoires reconnectables, avec des écarts temporels et des positions spatiales compatibles avec le critère de liaison d’AFLink. AFLink a donc un effet plus fort pour C2. Ce résultat soutient le choix retenu dans FiTAC-MOT.

4.3 Expérience 3 : comparaison avec d’autres modèles de suivi

4.3.1 Protocole expérimental

Cette expérience compare FiTAC-MOT avec ByteTrack [4] et BoT-SORT + FastReID [5, 7] sur `capfica4`. Les mêmes séquences de test et le même protocole TrackEval sont utilisés pour toutes les méthodes. Les trois traqueurs utilisent les mêmes boîtes de détection exportées par notre pipeline. Leur vitesse en ligne est mesurée après l’initialisation des modèles, sans exécuter le détecteur ni les projections gnomoniques. La génération de la vidéo, l’évaluation et AFLink sont également exclues du chronométrage.

4.3.2 Résultats

Méthode	HOTA	DetA	AssA	MOTA	IDF1	IDSW	Frag	Pred. IDs	FPS en ligne
ByteTrack	37.409	77.759	18.009	79.932	31.114	42	178	43	3179.715
BoT-SORT + FastReID	36.613	70.575	19.010	80.197	32.620	47	192	50	48.815
FiTAC-MOT	61.423	86.976	43.377	86.848	55.837	44	142	36	20.616

TABLE 8 – Comparaison entre le point de sauvegarde final de FiTAC-MOT et des traqueurs de référence sur `capfica4`, avec le protocole horizontal historique.

4.3.3 Discussion

FiTAC-MOT obtient les meilleurs résultats sur HOTA, DetA, AssA, MOTA et IDF1. Il produit aussi le plus faible nombre d’identités prédites et le plus faible nombre de fragments parmi les méthodes comparées. ByteTrack obtient le plus faible nombre de changements d’identité, avec 42 contre 44 pour FiTAC-MOT, mais ses scores d’association et d’identité restent plus bas. Comme notre objectif principal est la continuité d’identité dans les changements d’apparence, IDF1 et AssA sont les métriques les plus importantes pour interpréter ce résultat.

L’écart de vitesse s’explique par la complexité différente des méthodes. Dans cette configuration, ByteTrack réalise principalement une association géométrique sur CPU et n’extraît pas de caractéristiques ReID. BoT-SORT ajoute une extraction FastReID, tandis que FiTAC-MOT calcule les caractéristiques OSNet et FastReID ainsi que le score temporel pour les paires candidates.

4.4 Expérience 4 : généralisation à d’autres scènes

4.4.1 Protocole expérimental

Cette expérience évalue si le modèle temporel, conçu principalement pour les changements d’apparence et les changements de vêtements, garde un intérêt dans d’autres scènes fisheye. Contrairement aux expériences précédentes, l’entraînement et le test sont réalisés sur des séquences WEPD-TOF. Les résultats sont donc mesurés sur les scènes WEPD-TOF listées dans le Tableau 10. L’objectif est d’observer si cette stratégie d’association temporelle reste utile lorsque le contexte visuel change.

Nous utilisons les scènes WEPD-TOF disponibles pour construire les paires d’entraînement et pour tester le traqueur dans des environnements différents, comme les magasins, les bureaux.

Ensemble	Séquences WEPD-TOF
Entraînement	jewelry_store_1&2, tech_store, printing_store
Validation	kindergarten
Test	convenience_store, empty_store, exhibition, it_office, large_office_1&2

TABLE 9 – Séquences WEPD-TOF utilisées pour l’entraînement, la validation et le test.

Séquence WEPD-TOF	Description courte	Changement d’apparence
convenience_store	Scène fermée, 7 personnes, sans forte occlusion.	Non
empty_store	Scène fermée, 1 personne, avec occlusion.	Non
exhibition	Scène semi-fermée, plus de 20 personnes, occlusions partielles.	Une fois
it_office	Scène ouverte, 9 personnes, sans occlusion complète.	Non
large_office_1&2	Scène ouverte, 4 à 5 personnes, avec quelques occlusions.	Non

TABLE 10 – Séquences WEPD-TOF utilisées pour tester la généralisation.

4.4.2 Résultats

Séquence WEPDTOF	HOTA	DetA	AssA	MOTA	IDF1	IDSW	FPS
convenience_store	54.063	53.759	54.940	42.252	65.374	3	25.580
empty_store	42.761	42.761	42.761	11.173	52.959	0	48.004
exhibition	40.903	35.334	48.828	7.487	48.450	8	10.790
it_office	48.067	36.545	64.658	35.264	59.762	3	20.722
large_office	45.674	32.903	63.843	40.843	59.181	3	28.693
large_office_2	45.747	39.076	53.804	36.418	61.912	0	28.283
COMBINÉ	46.573	39.867	55.617	26.996	56.747	17	20.496

TABLE 11 – Évaluation de FiTAC-MOT sur plusieurs scènes WEPDTOF.

Méthode	HOTA	DetA	AssA	MOTA	IDF1	IDSW	FPS
ByteTrack	41.229	34.187	50.674	22.007	47.494	25	347.820
BoT-SORT + FastReID	40.981	34.244	50.212	22.076	46.948	27	64.470
FiTAC-MOT	46.573	39.867	55.617	26.996	56.747	17	20.496

TABLE 12 – Comparaison avec deux autres modèles sur le test WEPDTOF.

4.4.3 Discussion

Le modèle temporel entraîné sur WEPDTOF garde un intérêt dans plusieurs scènes différentes. Sur les six séquences, FiTAC-MOT obtient le meilleur HOTA, la meilleure AssA, le meilleur IDF1 et le plus faible nombre de changements d’identité parmi les trois méthodes comparées.

Si nous regardons des cas concrets, ce modèle fait également preuve d’une certaine capacité à traiter d’autres difficultés. Par exemple, dans la scène `empty_store`, où un piéton est caché puis réapparaît, le modèle est toujours capable de déterminer qu’il s’agit de la même personne.

Les résultats restent dépendants de la qualité des détections. La scène `exhibition` est plus difficile, avec plus de piétons et plus d’erreurs de détection, ce qui réduit la vitesse et la performance globale. Les scènes moins denses, comme `large_office` et `large_office_2`, donnent des changements d’identité plus faibles.

4.5 Évaluation complémentaire avec boîtes orientées

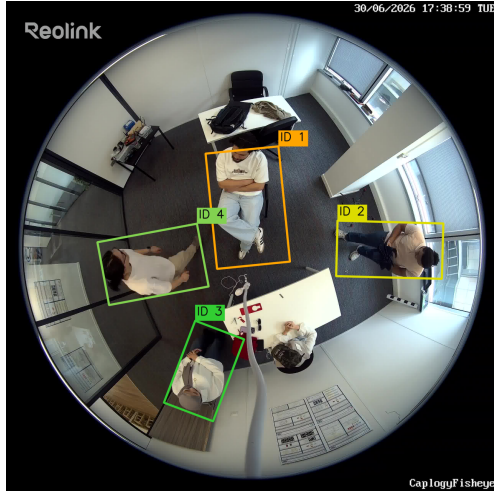
Le protocole orienté compare les polygones des boîtes prédites aux nouvelles annotations manuelles de **capfica4**. L'appariement CLEAR et Identity utilise un seuil d'IoU polygonale de 0.5. Le traqueur en ligne est évalué avant et après le post-traitement AFLink.

Configuration	rHOTA	rDetA	rAssA	rLocA	MOTA	IDF1	IDSW	IDs prédits
FiTAC-MOT sans AFLink	35.392	60.877	20.722	78.851	77.751	38.777	51	49
FiTAC-MOT avec AFLink	43.967	60.877	31.807	78.851	77.955	53.475	40	36

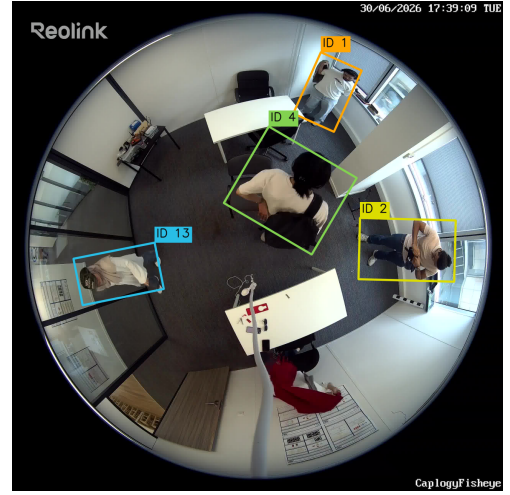
TABLE 13 – Évaluation du point de sauvegarde final sur les annotations orientées de **capfica4**.

Le protocole orienté produit une localisation moyenne rLocA de 78.851, ce qui indique que les boîtes appariées restent correctement alignées. AFLink ne modifie pas ces boîtes ni DetA, mais reconnecte leurs identifiants : rHOTA passe de 35.392 à 43.967, rAssA de 20.722 à 31.807 et IDF1 de 38.777 à 53.475, tandis que les changements d'identité diminuent de 51 à 40. Les scores restent différents de ceux du protocole horizontal, car l'IoU polygonale est plus stricte et la nouvelle vérité terrain contient 5380 boîtes, contre 5292 dans l'ancienne version. Cette évaluation ne remet donc pas directement en cause les comparaisons historiques ; elle fournit un protocole complémentaire, mieux adapté à la géométrie fisheye.

5 Visualisation qualitative des résultats



(a) Pipeline normal, boîtes orientées, trame 30.



(b) Pipeline normal, boîtes orientées, trame 130.



(c) Détections GT orientées, trame 30.



(d) Détections GT orientées, trame 130.

FIGURE 7 – Résultats qualitatifs de FiTAC-MOT sur `capfica4` avec des boîtes orientées. Les sous-figures (a) et (b) utilisent les détections automatiques, tandis que les sous-figures (c) et (d) utilisent les boîtes GT comme détections.

En prenant les sous-figures (c) et (d) comme références, les comportements des personnes dans la vidéo sont les suivants : ID 1 porte une veste marron ; ID 2 se prépare à ouvrir un parapluie ; ID 3 enroule une écharpe rouge sur sa tête ; ID 4 met son sac à dos ; ID 5 met une casquette, passe de la position assise à la position debout, puis commence à marcher.

6 Conclusion

Les expériences confirment l'intérêt de FiTAC-MOT pour le suivi de piétons en vidéos fisheye avec changements d'apparence. Dans le protocole horizontal historique, le point de sauvegarde final atteint un HOTA de 48.673, une AssA de 27.238 et un IDF1 de 40.245 sans post-traitement. Avec AFLink, ces valeurs atteignent respectivement 61.423, 43.377 et 55.837, tandis que le nombre de changements d'identité diminue de 55 à 44.

L'étude d'ablation montre que le modèle temporel, la récupération temporelle et AFLink jouent des rôles complémentaires. Dans l'évaluation orientée, AFLink augmente rHOTA de 35.392 à 43.967 et IDF1 de 38.777 à 53.475, tout en réduisant les changements d'identité de 51 à 40. Ces valeurs sont rapportées séparément, car le protocole polygonal est plus strict. Enfin, l'expérience sur WEPDToF indique que la stratégie temporelle garde un intérêt dans d'autres scènes fisheye.

Conclusion générale

7 Conclusion

Ce stage a porté sur le suivi de piétons dans des vidéos fisheye. Dans ce contexte, l'apparence d'une même personne peut changer fortement au cours d'une séquence. Les vêtements, la posture, l'orientation du corps et l'échelle apparente dans l'image modifient les vecteurs d'apparence. Ces variations peuvent provoquer de mauvaises associations entre détections et trajectoires, puis des changements d'identité.

Pour répondre à ce problème, nous avons proposé FiTAC-MOT, un traqueur construit dans un pipeline de type *suivi par détection*. La méthode conserve un coût d'apparence de référence basé sur OSNet, puis ajoute une branche temporelle fondée sur FastReID. Cette branche utilise une mémoire courte et une mémoire longue afin de comparer une détection courante avec l'historique d'une trajectoire. La fusion temporelle et le mécanisme de récupération temporelle permettent ensuite de corriger certaines associations difficiles lorsque l'apparence varie. Enfin, AFLink est utilisé comme étape finale pour reconnecter des fragments de trajectoires et réduire le nombre d'identités prédites.

Le travail a aussi nécessité la préparation des données expérimentales. Nous avons utilisé les scènes `jewelry` de WEPDToF comme données d'entraînement complémentaires. Nous avons également construit CapFiCA, un jeu de données fisheye contenant des changements d'apparence contrôlés. Les annotations ont été produites à partir des sorties d'un traqueur, puis corrigées manuellement dans notre éditeur GT.

Les expériences montrent que FiTAC-MOT améliore la stabilité des identités sur CapFiCA. L'étude d'ablation indique que le score temporel, la récupération temporelle et AFLink apportent des gains complémentaires. Avec le point de sauvegarde final, AFLink augmente l'IDF1 de 40.245 à 55.837 dans le protocole horizontal et de 38.777 à 53.475 dans le protocole orienté. L'évaluation complémentaire avec annotations orientées et IoU polygonale prend mieux en compte la géométrie fisheye, mais doit rester distincte du

protocole historique. L'étude de la mémoire longue montre enfin que sa perte auxiliaire diminue pendant l'entraînement, sans qu'une influence plus forte ou un réentraînement sur les nouvelles paires améliore les mesures de suivi. Le point de sauvegarde final décrit dans ce rapport a donc été conservé.

8 Perspectives

Plusieurs perspectives peuvent être envisagées à partir des limites observées pendant les expériences.

1. La fusion entre le score temporel et le coût de référence reste peu naturelle.
 - Entraîner un modèle de fusion unifié, qui produirait directement le coût d'association final ou une probabilité d'acceptation/refus.
 - Entraîner directement avec le résultat de l'association comme objectif, en apprenant à choisir la bonne trajectoire dans l'ensemble des candidats d'une même trame. Cette formulation serait plus proche de l'association de référence et pourrait à terme remplacer cette étape.
2. La contribution incrémentale de la mémoire long terme reste faible.
 - Utiliser davantage la mémoire longue lorsque la branche courte est incertaine. Augmenter son poids lorsque la mémoire courte distingue mal les candidats.
 - Apprendre directement la fusion entre les contextes court et long terme à partir du résultat d'association.
 - Redéfinir l'objectif de la mémoire longue : ne plus seulement vérifier si la détection ressemble à l'apparence moyenne de la mémoire longue, mais évaluer si cette détection reste cohérente avec la plage de variation à long terme de l'identité, ou si la tendance de changement observée est raisonnable.
3. Adapter dynamiquement l'espace ReID à la séquence courante.
 - S'inspirer de History-Aware MOT pour utiliser l'historique des trajectoires de la séquence courante et construire un espace d'apparence plus discriminant.
 - Réduire la dépendance au seul modèle appris sur les données d'entraînement, afin de mieux séparer les piétons visuellement proches dans une nouvelle vidéo.
4. Ajouter des attributs visuels explicites au modèle temporel.
 - Ajouter des attributs explicites au score temporel, comme la couleur des vêtements, les accessoires, l'orientation du corps ou la posture.

Bibliographie

- [1] Nicolai Wojke, Alex Bewley, and Dietrich Paulus. Simple online and realtime tracking with a deep association metric. In *2017 IEEE International Conference on Image Processing (ICIP)*, pages 3645–3649, 2017.
- [2] Alex Bewley, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Upcroft. Simple online and realtime tracking. In *2016 IEEE International Conference on Image Processing (ICIP)*, pages 3464–3468, 2016.
- [3] Yunhao Du, Zhicheng Zhao, Yang Song, Yanyun Zhao, Fei Su, Tao Gong, and Hongying Meng. Strongsort : Make deepsort great again. *IEEE Transactions on Multimedia*, 25 :8725–8738, 2023.
- [4] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. Bytetrack : Multi-object tracking by associating every detection box. In *European Conference on Computer Vision*, pages 20–37, 2022.
- [5] Nir Aharon, Roy Orfaig, and Ben-Zion Bobrovsky. Bot-sort : Robust associations multi-pedestrian tracking. *arXiv preprint arXiv :2206.14651*, 2022.
- [6] Hao Luo, Youzhi Gu, Xingyu Liao, Shenqi Lai, and Wei Jiang. Bag of tricks and a strong baseline for deep person re-identification. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2019.
- [7] Lingxiao He, Xingyu Liao, Wu Liu, Xinchun Liu, Peng Cheng, and Tao Mei. Fastreid : A pytorch toolbox for general instance re-identification. In *Proceedings of the 31st ACM International Conference on Multimedia*, pages 9664–9667, 2023.
- [8] Kaiyang Zhou, Yongxin Yang, Andrea Cavallaro, and Tao Xiang. Omni-scale feature learning for person re-identification. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 3702–3712, 2019.
- [9] Jiarui Cai, Mingze Xu, Wei Li, Yuanjun Xiong, Wei Xia, Zheng Tu, and Stefano Soatto. Memot : Multi-object tracking with memory. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8090–8100, 2022.

- [10] M. O. Tezcan, Z. Duan, M. Cokbas, P. Ishwar, and J. Konrad. Wepdtof : A dataset and benchmark algorithms for in-the-wild people detection and tracking from overhead fisheye cameras. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2022.
- [11] Jonathon Luiten, Aljosa Osep, Patrick Dendorfer, Philip Torr, Andreas Geiger, Laura Leal-Taixé, and Bastian Leibe. Trackeval : A code base for multi-object tracking evaluation. <https://github.com/JonathonLuiten/TrackEval>, 2020.
- [12] Jonathon Luiten, Aljosa Osep, Patrick Dendorfer, Philip Torr, Andreas Geiger, Laura Leal-Taixé, and Bastian Leibe. Hota : A higher order metric for evaluating multi-object tracking. *International Journal of Computer Vision*, 129 :548–578, 2021.
- [13] Ergys Ristani, Francesco Solera, Roger Zou, Rita Cucchiara, and Carlo Tomasi. Performance measures and a data set for multi-target, multi-camera tracking. In *European Conference on Computer Vision Workshops*, pages 17–35, 2016.
- [14] Keni Bernardin and Rainer Stiefelhagen. Evaluating multiple object tracking performance : The clear mot metrics. *EURASIP Journal on Image and Video Processing*, 2008 :246309, 2008.
- [15] Chien-Yao Wang, I-Hau Yeh, and Hong-Yuan Mark Liao. Yolov9 : Learning what you want to learn using programmable gradient information. *arXiv preprint arXiv :2402.13616*, 2024.

Annexes

9 Planning synthétique du stage

Le Tableau 14 résume l’avancement du stage par périodes d’une à deux semaines, du 9 février au 8 août 2026.

TABLE 14 – Planning synthétique du stage.

Tâche	Début	Fin	État
Recherche bibliographique et définition du sujet	09/02/2026	22/02/2026	Terminé
Étude de l’état de l’art et prise en main du pipeline	23/02/2026	08/03/2026	Terminé
Mise en place de la configuration de référence Deep-SORT/StrongSORT	09/03/2026	22/03/2026	Terminé
Premiers scores temporels et comparaisons exploratoires	23/03/2026	05/04/2026	Terminé
Préparation des paires détection–trajectoire	06/04/2026	19/04/2026	Terminé
Développement des mémoires courte et longue	20/04/2026	03/05/2026	Terminé
Entraînement et diagnostic du modèle temporel	04/05/2026	17/05/2026	Terminé
Intégration du traqueur dans le pipeline détection–suivi	18/05/2026	31/05/2026	Terminé
Création et correction des annotations CapFiCA	01/06/2026	14/06/2026	Terminé
Études d’ablation et comparaison des encodeurs	15/06/2026	28/06/2026	Terminé
Comparaison avec les méthodes de référence	29/06/2026	12/07/2026	Terminé
Réglage et validation de la configuration finale	13/07/2026	26/07/2026	Terminé
Évaluation orientée et analyse de la mémoire longue	27/07/2026	02/08/2026	Terminé
Finalisation du rapport, du README et de la reproductibilité	03/08/2026	08/08/2026	Terminé